A Major Project Report on

UART to Ethernet Conversion Module Using Kintex-7

submitted in partial fulfillment for award of the degree of Bachelor of Technology in

Electronics and Communication Engineering

by

K Sai Tejaswi	**16321A04C3**
V Sudhanvi Vakul	**16321A04F9**
Sweta Singh	**16321A04G8**

Under the esteemed Guidance of

Internal Guide

Ms S Deepika

Assistant Professor, ECE Department

Bhoj Reddy Engineering College for Women

Department of Electronics and Communications Engineering

(Sponsored by Sangam Laxmibai Vidyapeet, approved by AICTE & affiliated o JNTUH)

Vinaynagar, Santoshnagar X roads, Saidabad, Hyderabad – 500 059

Ph: +91-40-2459 2400 Fax: +91-40-2453 7281, www.brecw.ac.in,

principal@brecw.ac.in

2019- 2020

Bhoj Reddy Engineering College for Women
(Sponsored by Sangam Laxmibai Vidyapeet, approved by AICTE & affiliated to JNTUH)
Vinaynagar, I S Sadan X Roads, Saidabad, Hyderabad 500 059, Telangana, India
Telephone: 040-2453 7282, Website: www.brecw.ac.in, Email: principal@brecw.ac.in

CERTIFICATE

This is to certify that the Major Project Report entitled **"UART to Ethernet Conversion Module Using Kintex-7"** is a bonafide work carried out by

K Sai Tejaswi	**16321A04C3**
V Sudhanvi Vakul	**16321A04F9**
Sweta Singh	**16321A04G8**

in partial fulfillment for award of the degree of Bachelor of Technology in Electronics and Communication Engineering from Bhoj Reddy Engineering College for Women, Hyderabad, affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH).

S Deepika	**N Shribala**
Assistant Professor	**HoD - ECE**

Acknowledgement

The satisfaction that accompanies the successful completion of the task would be incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown all the efforts with success.

We would like to express my sincere gratitude to **S Deepika**, Assistant professor and Project guide for the eminent guidance and supervision at every stage.

We are thankful to **Ms. N.Shribala**, Head of the Department for her valuable guidance, encouragement during my Project.

We also thank **Dr. J Madhavan**, Principal, BRECW for providing the wonderful education environment in our college.

We are equally thankful to **Ms. S Manjula**, Project Co-ordinator for her continuous support and all the staff of Electronics and Communication Department of BRECW for their timely help and suggestions in the Project.

K Sai Tejaswi(16321A04C3) tejakommareddy1313@gmail.com
V Sudhanvi Vakul(16321A04F9) sudhanvi.vakul.tx.rx@gmail.com
Sweta Singh(16321A04G8) singhsweta0904@gmail.com

Contents

		Page No
List of Figures		i
List of Tables		iii
Abbreviations		iv
Abstract		vii
Chapter 1	Introduction	1
1.1	Introduction	1
1.2	Aim of the Project	2
1.3	Objective of the Project	2
1.4	Motivation of the Project	2
1.5	Literature Survey	3
1.6	Technical Approach	3
1.7	Organisation of the Report	4
1.8	Conclusion	5
Chapter 2	Hardware Description	6
2.1	Introduction	6
2.2	Features of Kintex-7 KC705 Board	6
2.3	Feature Description	8
2.3.1	Kintex-7 FPGA	8
2.3.2	DDR3 Memory Module	9
2.3.3	Linear BPI Flash Memory	9
2.3.4	Quad SPI Flash Memory	10
2.3.5	SD card Interface	10
2.3.6	USB JTAG Module	11
2.3.7	Clock Generation	11
2.3.8	GTX Transceiver	15
2.3.9	PCI Express Edge Connector	15
2.3.10	SFP/SFP+ Connector	16
2.3.11	10/100/1000 Tri-Speed Ethernet PHY	17
2.3.12	SGMII GTX Transceiver Clock Generation	18
2.3.13	USB-to-UART Bridge	18
2.3.14	HDMI Video Output	19

2.3.15 LCD Character Display 19
2.3.16 I2C Bus Swicth 19
2.3.17 Status LEDs 20
2.3.18 Ethernet PHY Status LEDs 20
2.3.19 User I/O 21
2.3.20 Switches 21
2.3.21 FPGA Mezzanine Card Interface 21
2.3.22 Power Management 22
2.3.23 XADC Header 22
2.4 Ethernet 802.3 Embedded System 22
2.4.1 Introduction 22
2.4.2 Network Model 23
2.4.3 Hardware 24
2.4.4 The Ethernet Media Access Controller(MAC) 25
2.4.5 The Ethernet Physical Layer(PHY) 25
2.4.6 Addressing 26
2.4.7 Half Duplex and Full Duplex Modes of Ethernet 27
2.4.8 MII Interface 27
2.4.9 MDIO Interface 28
2.4.10 Ethernet Frame 28
2.5 UART 29
2.5.1 Introduction 29
2.5.2 Synchronous Serial Transmission 31
2.5.3 Asynchronous Serial Transmission 32
2.5.4 Bits, Baud and Symbols 32
2.5.5 Asynchronous Serial Reception 33
2.6 Conclusion 33
Chapter 3 IP Cores 34
3.1 Introduction 34
3.2 IP Cores 34
3.3 List of IP cores used 34
3.4 MicroBlaze Processor 35
3.4.1 Introduction 35

3.4.2 Architecture 36
3.4.3 Features 37
3.4.4 Instructions 37
3.4.5 Registers 37
3.4.6 Pipeline 38
3.5 MicroBlaze Debug Module 38
3.6 Processor System Reset 39
3.6.1 Features 39
3.6.2 Module Circuit Description 39
3.7 AXI Interconnect 40
3.8 AXI Interrupt Controller(INTC) 41
3.8.1 Features 41
3.8.2 Blocks 41
3.9 AXI UART Lite 42
3.9.1 Block Diagram 42
3.9.2 Features 43
3.10 AXI Ethernet Lite 43
3.10.1 Transmitter 44
3.10.2 Receiver 44
3.10.3 Features 44
3.11 AXI BRAM Controller 45
3.11.1 BRAM Interface 45
3.11.2 Supported Memory Size 45
3.11.3 Features 45
3.11.4 Application 46
3.12 Block Memory Generator 46
3.12.1 Features 46
3.12.2 Selectable Memory Algorithm 47
3.13 Clocking Wizard 47
3.14 Concat 48
Chapter 4 Software Description 49
4.1 Introduction 49
4.2 Xilinx Vivado ISE Design Suite 49

4.2.1 Xilinx ISE 49
4.2.2 Xilinx Vivado 49
4.2.3 User Interface 50
4.2.4 Features and Components 50
4.2.5 Design Support 52
4.3 WireShark 53
4.3.1 Introduction 53
4.3.2 Features 55
4.3.3 Security 55
4.4 Using Vivado IDE and WireShark 56
4.4.1 Invoke the Vivado IDE and Create a Project 56
4.4.2 Create and IP Integrator Design 57
4.4.3 Use the Board Tab to Connect the BoardInterfaces 58
4.4.4 Connect the IPCores 59
4.4.5 Validate Block Design 60
4.4.6 Synthesise the Design 60
4.4.7 Take the Design through Implementation 60
4.4.8 Exporting the Design to SDK 61
4.5 Conclusion 63
Chapter 5 Implementation 64
5.1 Introduction 64
5.2 Block Diagram & Circuit Diagram 64
5.3 Flow Chart & Algorithm 66
5.4 Working Principle 69
5.4.1 UART to Ethernet Conversion Mechanism 69
5.4.2 Ethernet to UART Conversion Mechanism 70
5.5 Advantages and Application 70
5.5.1 Advantages 70
5.5.2 Applications 71
5.6 Conclusion 72
Chapter 6 Results and Discussions 73
6.1 Introduction 73
6.2 Ethernet to Serial Conversion Output 73

6.3 Serial to Ethernet Conversion Output 74
6.4 Conclusion 77
Chapter 7 Conclusion and Future Scope 78
7.1 Conclusion 78
7.2 Future Scope 79
References 80

List of Figures

		Page No.
Figure 2.1	KC705 Block Diagram	7
Figure 2.2	KC705 Board Components	8
Figure 2.3	System Clock Source	12
Figure 2.4	User Clock Source	13
Figure 2.5	User SMA Clock Source	13
Figure 2.6	GTX SMA Clock Source	14
Figure 2.7	PCI Express Clock	16
Figure 2.8	PCI Express Lane Size Select Jumper J32	16
Figure 2.9	LCD Display	19
Figure 2.10	Ethernet PHY Status LEDs	20
Figure 2.11	Comparison between OSI and TCP/IP Model	24
Figure 2.12	PHY and MAC Layer 100-Mbit Network	24
Figure 2.13	Ethernet Link	25
Figure 2.14	Theoretical PHY Block Diagram in 100Base-TX Mode	25
Figure 2.15	Description of MII	27
Figure 2.16	Ethernet Frame Structure	28
Figure 2.17	UART Block Diagram	30
Figure 2.18	Serial Transmission	31
Figure 2.19	Asynchronous Serial Transmission	31
Figure 2.20	Asynchronous Serial Reception	33
Figure 3.1	MicroBlaze Core Diagram	36
Figure 3.2	Block Diagram of Processor System Reset Module	40

Figure 3.3	Block Diagram of AXI UART Lite	42
Figure 3.4	Concat IP in a System	48
Figure 4.1	Vivado Simulator	51
Figure 4.2	Vivado IP Integrator	51
Figure 4.3	Vivado TCL Shell	52
Figure 4.4	Overview of Design Flow	53
Figure 4.5	Packets Received via Ethernet	54
Figure 4.6	Vivado Quick Start Page	56
Figure 4.7	Name and Block Design	57
Figure 4.8	Export Hardware Dialog Box	61
Figure 4.9	Status Dialog Box	62
Figure 5.1	Block Diagram	64
Figure 5.2	Circuit Diagram	65
Figure 5.3	Serial to Ethernet Connection establishment Flowchart	66
Figure 5.4	Ethernet to Serial Connection establishment Flowchart	68
Figure 5.5	Application 1	71
Figure 5.6	Application 2	71
Figure 5.7	Application 3	71
Figure 5.8	Application 4	71
Figure 6.1	Creating a COM Port	73
Figure 6.2	Tera Term Serial Port Setup Window	74
Figure 6.3	Message Received via UART in Tera Term	74
Figure 6.4	WireShark Network Analyzer	75
Figure 6.5	Packets Captured via Ethernet	75

List of Tables

		Page No.
Table 2.1	KC705 Board Clock Sources	11
Table 2.2	PHY Default Interface Mode	18
Table 2.3	Status LEDs	20
Table 2.4	Ethernet PHY Status LEDs	21
Table 3.1	IP Cores	35

Abbreviations

		Page No.
UART	Universal Asynchronous Receive/Transmit	1
IC	Integrated Circuit	1
SPI	Serial Peripheral Interface	1
I^2C	Inter Integrated Circuits	1
USB	Universal Serial Bus	1
LAN	Local Area Network	1
WAN	Wide Area Network	1
RS-232	Recommended Standard Number 232	1
TCP	Transmission Control Protocol	1
UDP	User Datagram Protocol	1
LMB	Local Memory Bus	2
SDK	Software Development Kit	2
COMM	Communication	2
JTAG	Joint Test Action Group	2
IP	Intellectual Property	3
FPGA	Field Programmable Gate Array	3
HDL	Hardware Description Language	4
DDR	Double Data Rate	6
SODIMM	Small Outline Dual In-line Memory Module	6
PCIe	Peripheral Component Interconnect Express	6
PHY	Physical Layer	6
I/O	Input/Output	6
BPI	Byte Peripheral Interface	6
SD	Secure Digital	6
GTX	Gigabit Transceiver	7
SFP	Small Factor Pluggable	7
HDMI	High Definition Multimedia Interface	7
LED	Light Emitting Diode	7

XADC	Analog to Digital Converter (X-added by Xilinx)	7
SDRAM	Synchronous Dynamic Random-Access Memory	9
BGA	Ball Grid Array	9
DIP	Dual In-line Package	10
SPST	Single Pole Single Throw	11
LVDS	Low Voltage Differential Signalling	11
MRCC	Multiple Region Clock Capable	12
PPM	Parts Per Million	12
AGP	Accelerated Graphic Port	15
SSD	Solid State Drive/Device	15
Wi-Fi	Wireless Fidelity	15
AER	Advanced Error Reporting	15
MAC	Media Access Control	15
TSMAC	Tri-Speed Media Access Controller	15
MII	Media Independent Interface	17
GMII	Gigabit Media Independent Interface	17
RGMII	Reduced Gigabit Media Independent Interface	17
SGMII	Serial Gigabit Media Independent Interface	17
TX	Transmit	18
RX	Receive	18
RTS	Request to Send	18
CTS	Clear to Send	18
VSync	Vertical Synchronization	19
HSync	Horizontal Synchronization	19
LCD	Liquid Crystal Display	19
ANSI	American National Standards Institute	20
GUI	Graphical User Interface	21
OSI	Open Systems Interconnect	23
FTP	File Transfer Protocol	23
HTTP	Hyper Text Transfer Protocol	23
SNTP	Simple Network Time Protocol	23

DNS	Domain Name System	23
NRZI	Non-Return to Zero/Inverted	24
IEEE	Institute of Electrical and Electronics Engineer	26
SFD	Start of Frame Delimiter	29
FCS	Frame Check Sequence	29
ESD	End of Stream Delimiter	29
FIFO	First in First out	29
MSR	Machine Status Register	38
EAR	Exception Address Register	38
ESR	Exception Status Register	38
FSR	Floating point Status Register	38
MDM	MicroBlaze Debug Module	38
AXI	Advanced eXtensible Interface	39
INTC	Interrupt Controller	41
AMBA	Advanced Microcontroller Bus Architecture	42
CRC	Cyclic Redundancy Check	44
BRAM	Block Random-Access Memory	45
ISE	Integrated Synthesis Environment	47
ARM	Advanced RISC Machine	53
AMS	Analog Mixed Signal	53

Abstract

A serial to Ethernet converter is a small electronic device which can convert Ethernet IP/TCP packets to either RS232, RS485 or RS422 serial data signals and vice versa. Some legacy products may only contain a serial port and accessing such products is increasingly becoming a challenge due to the inability to add multiple such products to a shared network and the inability to access the legacy product from long distances. A serial-to-Ethernet converter provides a simple solution to solve the above challenges

Data acquisition is the process of sampling signals that measure real world physical conditions and converting the resulting samples into digital numeric values that can be manipulated by a computer. The communication between them is standard asynchronous serial form by typical serial control card, which have complicated cable wiring, frequently interruption of CPU, which greatly reduces the CPU's efficiency and impacts the system's real-time processing. To resolve these problems, a method of multi-serials to Ethernet Gateway based on the field programmable gate array (FPGA) with network interface chip is implemented in this project. The Gateway will send data as Ethernet frame format after receiving serial data, indirectly achieves serial communication, simplifies cabinet wiring and improves CPU's efficiency. In this project we will be using KIntex KC705 board. On board consists of UART and Ethernet interfaces. The software to make connections enable on board is Vivado which comes with Software Development Kit (SDK). The FPGA logic is implemented to read data from serial input ports. The KC705 takes the input from the serial port and send to PC in Ethernet form. In PC Wireshark application is used to read the data from Ethernet. The code is developed in VHDL

Chapter 1

Introduction

1.1 Introduction

UART stands for Universal Asynchronous Receiver/Transmitter. Universal asynchronous receiver-transmitter is one of the simplest and oldest forms of device-to-device digital communication. One can find UART devices as a part of integrated circuits (ICs) or as individual components. UARTs communicate between two separate nodes but it's not a communication protocol like SPI and I2C, but a physical circuit in a microcontroller, or a stand-alone IC. A UART's main purpose is to transmit and receive serial data. While USB has almost completely replaced those old cables and connectors, UARTs are definitely not a thing of the past.

Ethernet is a technology that connects wired local area networks (LANs) and enables the device to communicate with each other through a protocol which is the common network language. This LAN is a network of computers and other electronic devices which covers a small area in our places like in the office, house, room or building. Unlike LAN, wide area network (WAN) covers much larger geographical areas. Furthermore, Ethernet is a protocol that controls the processes on how the data is transmitted through LAN. It also indicates how the network devices can transmit and format data packets so that the other network devices in the same area network segment can be able to receive, process and recognize them.

Though Ethernet communication is very common in a variety of devices ranging from tiny embedded devices to high-performance personal computers, plenty of legacy devices with no Ethernet connectivity are still in use in many fields. Such legacy devices are usually equipped with traditional serial communication interfaces such as RS-232 or RS-485. To enable existing legacy devices with such an interface to connect to an Ethernet network, it is required to upgrade the communication interface of the legacy devices. This may cause both considerable cost and time or be technically impossible. UART-to-Ethernet converters are devices for converting TCP/UDP packets into asynchronous serial data and vice versa. These devices can help solve the problem without the need for modifying the existing legacy devices.

The UART-to-Ethernet converter bridges seamlessly asynchronous serial ports of the legacy devices and the Ethernet network and the conversion of data takes place.

1.2 Aim of the Project

Communication is an important management function closely associated with all other managerial functions. Be it actual human beings, or devices operated by human beings, establishing a communication is extremely vital. Along with the initiation of a secure and an error free communication, it should also be kept in mind that this communication is useful and serviceable only if both ends are able to comprehend the information transmitted i.e. both end points should be compatible to each other. Keeping in mind the same principle, in this project, we aimed to establish a communication between two leading and widely used devices today- the UART and the Ethernet. Our project intends to achieve a two-way communication between these two devices.

1.3 Objective of the Project

The task was to design a system which transmits and converts data transmitted from UART to that comprehendible by the Ethernet, and vice versa, hence, the project originated from designing a circuit on Vivado, Xilinx. The brain of the circuit was the MicroBlaze soft core processor which came with a local bus memory (LMB) for storage of data. The project also utilizes the SDK (which is also a part of Xilinx software) to test all the peripherals present in the design using several programs written using Embedded C language, the debugging and running of which generated the main results in WireShark (for UART to Ethernet data conversion) and Tera Term (for Ethernet to UART data conversion), with the help of several COMM ports and also JTAG.

1.4 Motivation of the Project

When a legacy product only contains a serial port for configuration or control, continuing to access the legacy product through the serial interface can become challenging over time. The following are the challenges of using a serial port:

• The inability to add multiple such products to a shared network,

• The inability to access the legacy product from long distances,

• Modern PCs generally do not have RS-232 ports.

Using Ethernet in place of the serial port provides the following benefits:

• The legacy product can be more easily shared (instead of changing a cable connection, a new connection over the existing network is made)

• Ability to access from longer distances. Cable length is no longer an issue

• Ethernet is a more common interface available on today's computing equipment

Moreover, both UART and the Ethernet are rapidly emerging technologies which have been widely adapted by people belonging to several different professions. The presence of a device or a method to easily generate an understanding or compatibility of data transmitted and received by the two, that too in both directions is undoubtedly very handy and will serve many purposes in the future in the field of embedded systems and communications.

1.5 Literature Survey

To do this project, we mainly relied on various reference guides and tutorials provided by Xilinx, which provided us detailed information regarding our board, i.e. the Kintex-7 KC705 Evaluation Board and the IP cores we used in the block design, their features and their limitations. The Xilinx user guides and manuals explaining how to use its Software Development Kit were also referred.

We also referred "An Implementation of a Configurable UART-to-Ethernet Converter", a research paper written by Jungho Moon and Myunggon Yoon, and published in the International Journal of Electronics and Communication Engineering (Vol:11, No:12, 2017). This paper, along with several websites helped us understand what actually happens and needs to happen when data is being transmitted from a serial device to Ethernet.

1.6 Technical Approach

In this project, we used Kintex-7 KC705 Evaluation Board provided by the Xilinx. The Kintex-7 KC705 board provides a hardware environment for developing and evaluating designs. We made use of the MicroBlaze soft core processor. Xilinx provides a wide selection of IP that is optimized for Xilinx FPGAs. Intellectual Property (IP) refers to pre-configured logic functions that can be used in our

design. We made use of the MicroBlaze soft core processor, which is highly configurable, allowing us to select a specific set of features required by our design. It is implemented with a Harvard memory architecture and has several commands regulating its data and instructions.

We also used the SDK provided by the Xilinx to debug and run the code which verifies the working of all the peripherals present in our design. Once this testing was done, we moved on another software, i.e. WireShark which we used to capture the Ethernet packets. For verifying the results of Ethernet to UART flow, we used TeraTerm.

In all, the three software we used in our project were Xilinx, WireShark and Tera Term. Xilinx is a software tool produced by Xilinx for synthesis and analysis of HDL designs, enabling the developer to synthesize ("compile") their block designs, perform timing analysis, examine RTL diagrams, simulate a design's reaction to different stimuli, generate the necessary bit file and HDL wrapper, and configure the target device with the programmer. Wireshark is the world's leading network traffic analyser, and an essential tool for any security professional or systems administrator. This free software lets us analyse network traffic in real time, and is often the best tool for troubleshooting issues on our network.

1.7 Organization of the Report

Chapter 1 deals with introduction to the project i.e., its aim, objective, motivation, literature survey and technical approach. Chapter 2 gives detailed information regarding the Kintex-7 KC705 board, listing out it’s features and summarizing them. The proceeding topics discuss the two prominent devices the two prominent devices used in our project, the UART and the Ethernet, explicating the background information needed to understand and regulate the information flow between the two. Chapter 3 begins with explaining what IP cores are and how they’re beneficial and cost friendly and later gives details regarding the many IP cores utilized in this project. Chapter 4 revolves around the two main software used in this project, i.e. Xilinx (ISE and SDK) and WireShark. The algorithm and the flowchart of the project is discussed in Chapter 5 and also enumerates the various steps taken together to obtain desired output. In Chapter6, results are given, which are also pictorially illustrated. The report concludes with Chapter 7, which also briefly analyses the future scope of the project.

1.8 Conclusion

This chapter gives the main gist and introduction to the project by discussing important aspects of the project like its aim, motivation behind choosing this project, objective and technical approach, along with the overall organization of this report.

Chapter 2

Hardware Description

2.1 Introduction

This chapter provides a detailed description of the Kintex-7 KC705 Evaluation Board, the Ethernet and the UART.

The KC705 evaluation board for the Kintex-7 FPGA provides a hardware environment for developing and evaluating designs targeting the Kintex-7 XC7K325T-2FFG900C FPGA. The KC705 board provides features common to many embedded processing systems, including a DDR3 SODIMM memory, an 8-lane PCI Express interface, a tri-mode Ethernet PHY, general purpose I/O, and a UART interface.

Ethernet is a way of connecting computers together in a local area network or LAN. It has been the most widely used method of linking computers together in LANs since the 1990s. The basic idea of its design is that multiple computers have access to it and can send data at anytime.

UART stands for Universal Asynchronous Receiver/Transmitter. It's not a communication protocol like SPI and I2C, but a physical circuit in a microcontroller, or a stand-alone IC. A UART's main purpose is to transmit and receive serial data.

2.2 Features of Kintex-7 KC705Board

The Kintex-7 KC705 board incorporates the following listed features: -

• Kintex-7 XC7K325T-2FFG900CFPGA

• 1 GB DDR3 memory SODIMM

• 128 MB Linear Byte Peripheral Interface (BPI) flash memory

• 128 Mb Quad Serial Peripheral Interface (SPI) flash memory

• Secure Digital (SD)connector

• USB JTAG via Digilent module

• Clock generation

• GTX transceivers

• PCI Express endpoint connectivity

• SFP+ Connector

• 10/100/1000 tri-speed Ethernet PHY

• USB-to-UART bridge

•High-Definition Multimedia Interface ™ (HDMI) technology codec

• I2Cbus

• Status LEDs

• User I/O

• Switches

• Power management

• XADC header

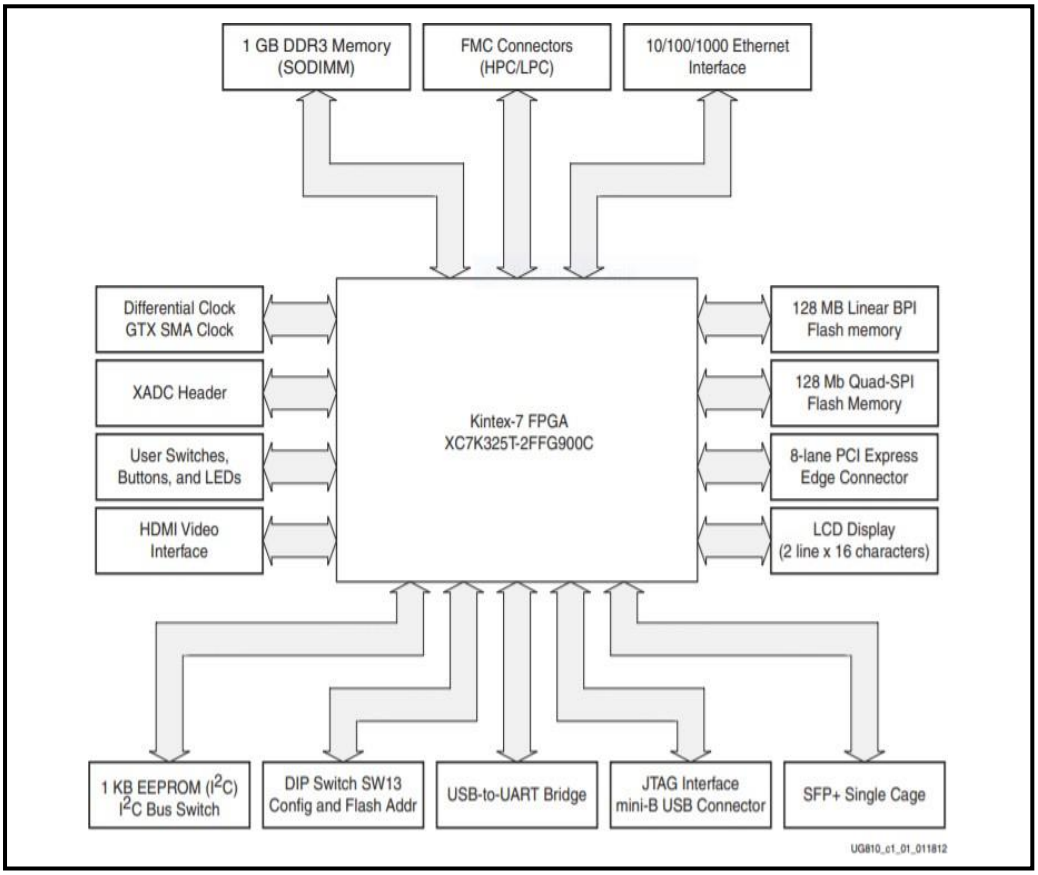

Figure 2.1: KC705 Board Diagram

The figure given above is an illustration of the features or components present in the Kintex-7 KC705 evaluation board. However, actual position of these features or components on the actual board may differ from what is being depicted in the figure. It may also vary from board to board.

2.3 Feature Descriptions

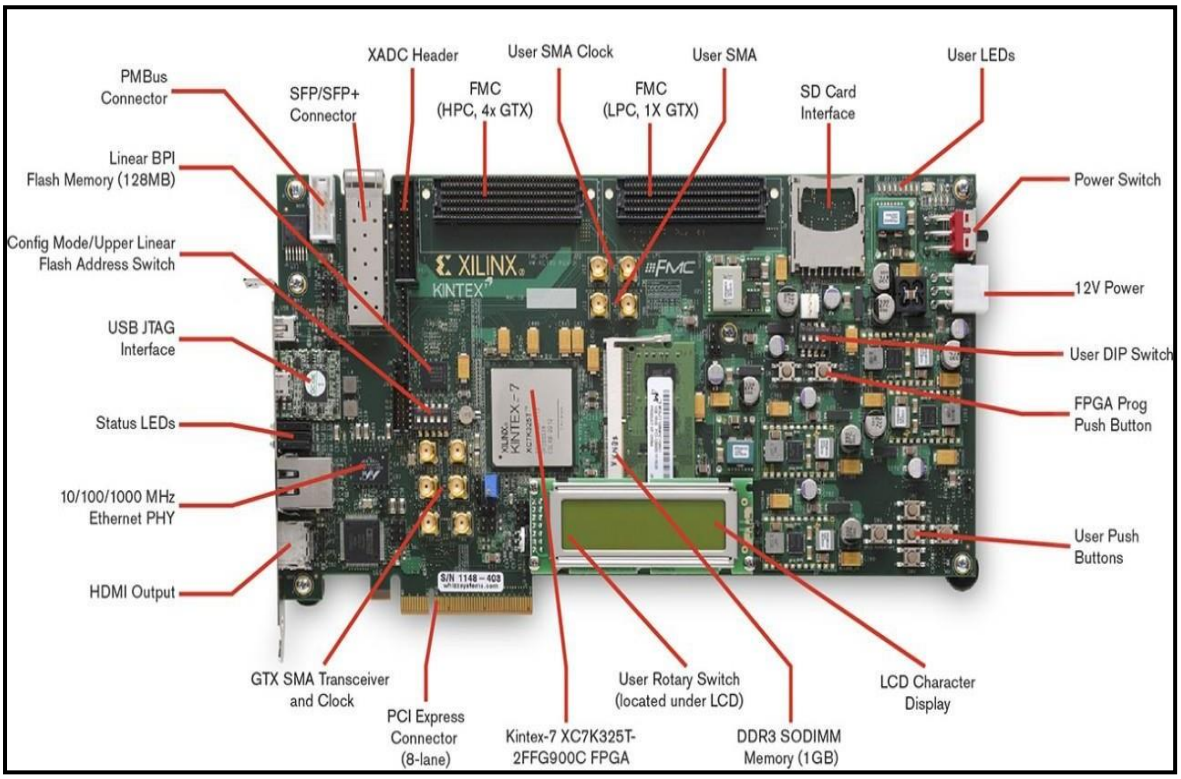

Figure 2.2: KC705 Board Components

2.3.1 Kintex-7FPGA

The KC705 board is populated with the Kintex-7 XC7K325T-2FFG900C FPGA. The KC705 board supports three of the five 7 series FPGA configuration modes:

• Master SPI flash memory using the onboard Quad SPI flash memory

• Master BPI flash memory using the onboard Linear BPI flash memory

• JTAG using a standard-A to micro-B USB cable for connecting the host PC to the KC705 board configuration port

2.3.2 DDR3 Memory Module

This memory module is a 1 GB DDR3 small outline dual-inline memory module (SODIMM). It provides volatile synchronous dynamic random-access memory (SDRAM) for storing user code and data.

• Part number: MT8JTF12864HZ-1G6G1 (Micron Technology)

• Configuration: 1GB (128 Mb x64)

• Supply voltage:1.5V

• Data path width: 64 bits

• Data rate: Up to 1,600MT/s

The DDR3 interface is implemented across I/O banks 32, 33, and 34. Each bank is a 1.5V high-performance (HP) bank.

An external 0.75V reference VTTREF is provided for data interface banks 32 and 34. Any interface connected to these banks that requires a reference voltage must use this FPGA voltage reference. The KC705 DDR3 SODIMM interface is a 40Ω impedance implementation.

2.3.3 Linear BPI Flash Memory

The Linear BPI flash memory located at U58 provides 128 MB of non-volatile storage that can be used for configuration or software storage. The data, address, and control signals are connected to the FPGA. The BPI flash memory device is packaged in a 64-pin BGA.

• Part number: PC28F00AP30TF(Micron)

• Supply voltage:2.5V

• Data path width: 16 bits (26 address lines and 7 control signals)

• Data rate: Up to 33MHz

The Linear BPI flash memory can synchronously configure the FPGA in Master BPI mode at the 33 MHz data rate supported by the PC28F00AP30TF flash memory. The fastest configuration method uses the external 66 MHz oscillator connected to the FPGA EMCCLK pin with a bitstream that has been built to divide

the configuration clock by two. The division is necessary to remain within the synchronous read timing specifications of the flash memory.

Multiple bitstreams can be stored in the Linear BPI flash memory. The two most significant address bits (A25, A24) of the flash memory are connected to DIP switch SW13 positions 1 and 2 respectively, and to the RS1 and RS0 pins of the FPGA.

2.3.4 Quad SPI Flash Memory

The Quad SPI flash memory located at U7 on the back side of the board provides 128 Mb of non-volatile storage that can be used for configuration and data storage.

• Part number: Micron N25Q128A13BSF40F or MT25QL128ABA8ESF-0SIT

• Supply voltage:2.8V

• Data path width: 4 bits

• Data rate: Various depending on Single/Dual/Quad mode and CLK rate

Four data lines and the FPGA CCLK pin are wired to the Quad SPI flash memory. A common chip selects (FPGA_FCS) shared between the Linear BPI flash memory and the Quad SPI flash memory is controlled by the configuration mode settings on DIP switch SW13 position 5 (M0) and a one-of-two demultiplexer device. If mode pin M0 = 1, the SPI flash memory device is selected. If mode pin M0 = 0, the Linear BPI flash memory device is selected.

2.3.5 SD Card Interface

The KC705 board includes a secure digital input/output (SDIO) interface to provide user-logic access to general purpose non-volatile SDIO memory cards and peripherals. The SD card slot is designed to support 50 MHz high speed SD cards. The SDIO signals are connected to I/O bank 12 which has its VCCO set to VADJ. A Texas Instruments I TXB0108 8-bit bidirectional voltage-level translator is used between the FPGA and the SD card connector.

2.3.6 USB JTAG Module

JTAG configuration is provided through a Digilent onboard USB-to-JTAG configuration logic module (U59) where a host computer accesses the KC705 board JTAG chain through a standard-A plug (host side) to micro-B plug (KC705 board side) USB cable.

When an FMC card is attached to the KC705 board, it is automatically added to the JTAG chain through electronically controlled single-pole single-throw (SPST) switches U76 and U77.The JTAG connectivity on the KC705 board allows a host computer to download bitstreams to the FPGA using the Xilinx software. In addition, the JTAG connector allows debug tools or a software debugger to access the FPGA. The Xilinx software tool can also indirectly program the Linear BPI or the Quad SPI flash memory. To accomplish this, the software configures the FPGA with a temporary design to access and program the BPI or Quad SPI flash memory device.

2.3.7 Clock Generation

The following table lists the five clock sources available for the FPGA fabric on the KC705 board: -

Table2.1: KC705 Board Clock Sources

Clock Name	Reference	Description
System Clock	U6	SiT9102 2.5V LVDS 200 MHz Fixed Frequency Oscillator (Si Time).
User Clock	U45	Si570 3.3V LVDS I2C Programmable Oscillator (Silicon Labs). Default power-on frequency 156.250 MHz.
User SMA Clock (differential pair)	J11	USER_SMA_CLOCK_P (net name).
	J12	USER_SMA_CLOCK_N (net name)
GTX SMA REF Clock (differential pair)	J16	SMA_MGT_REFCLK_P (net name).
	J15	SMA_MGT_REFCLK_N (net name).
Jitter Attenuated Clock	U70	Si5324C LVDS precision clock multiplier/jitter attenuator (Silicon Labs).

System Clock Source

The KC705 board has a 2.5V LVDS differential 200 MHz oscillator (U6) soldered onto the back side of the board and wired to an FPGA MRCC clock input on bank 33. This 200 MHz signal pair is named SYSCLK_P and SYSCLK_N, which are connected to FPGA U1 pins AD12 and AD11 respectively.

• Oscillator: Si Time SiT9102AI-243N25E200.00000 (200MHz)

• PPM frequency jitter: 50 ppm

• Differential Output

The system clock circuit is shown in the figure below: -

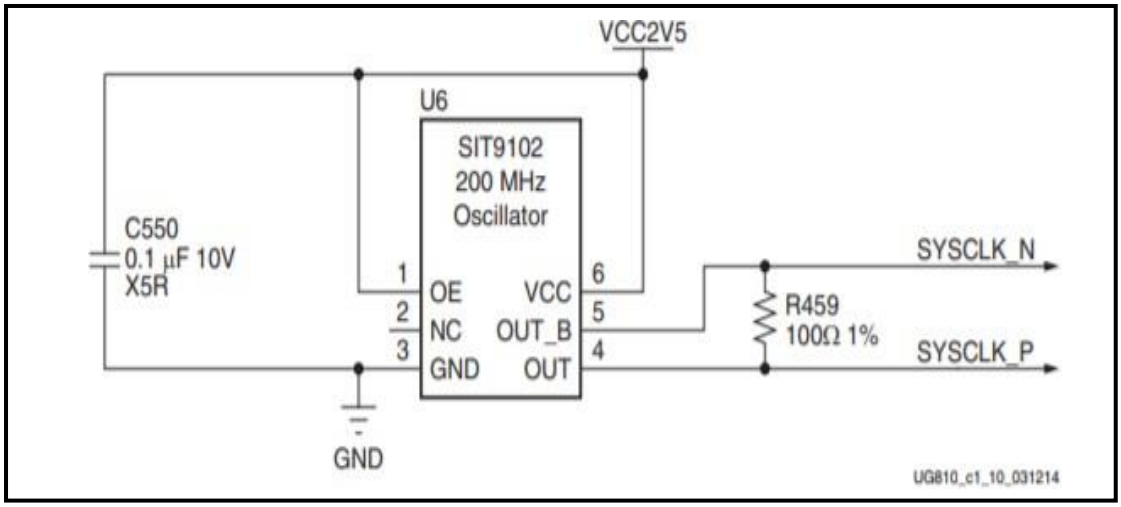

Figure 2.3: System Clock Source

Programmable User Clock Source

The KC705 board has a programmable low-jitter 3.3V differential oscillator (U45) the FPGA MRCC inputs of bank 15. This USER_CLOCK_P and USER_CLOCK_N clock signal pair are connected to FPGA U1 pins K28 and K29 respectively. On power-up the user clock defaults to an output frequency of 156.250 MHz. User applications can change the output frequency within the range of 10 MHz to 810 MHz through an I2C interface. Power cycling the KC705 board reverts the user clock to its default frequency of 156.250MHz.

• Programmable Oscillator: Silicon Labs Si570BAB0000544DG (10 MHz - 810 MHz)

• Differential Output

The user clock circuit is shown in the figure below: -

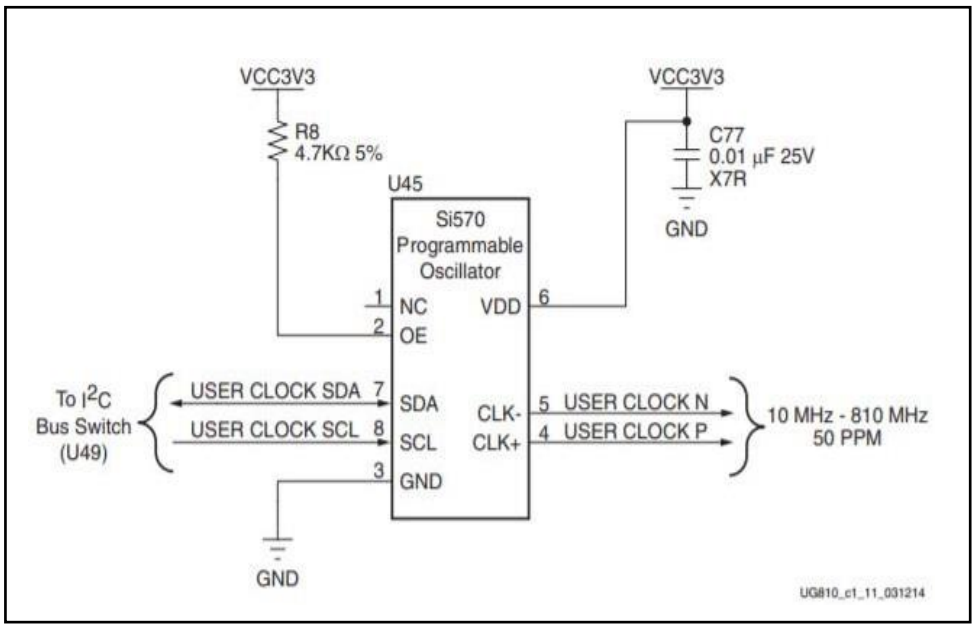

Figure 2.4: User Clock Source

User SMA Clock Input

An external high-precision clock signal can be provided to the FPGA bank 15 by connecting differential clock signals through the onboard 50Ω SMA connectors J11 (P) and J12 (N). The differential clock has signal names are USER_SMA_CLOCK_P and USER_SMA_CLOCK_N, which are connected to FPGA U1 pins L25 and K25, respectively. J11 (P) and J12 (N) are connected directly to the noted FPGA pins (no series capacitors and no external parallel termination resistor). The user-provided 2.5V differential clock circuit is shown in the figure below:-

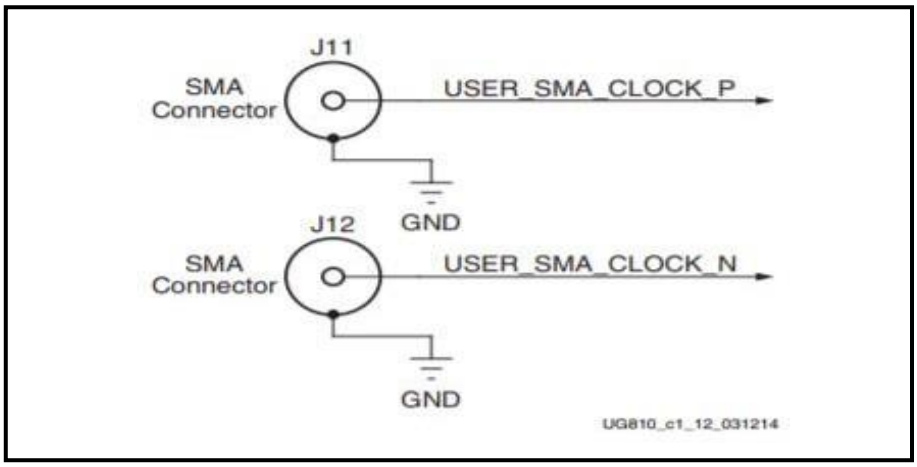

Figure 2.5: User SMA Clock Source

GTX SMA Clock Input

The KC705 board includes a pair of SMA connectors for a GTX clock wired to GTX Quad bank 117. The GTX SMA clock is differential in nature. This differential clock has signal names SMA_MGT_REFCLK_P and SMA_REFCLK_N, which are connected to FPGA U1 pins J8 and J7 respectively.

• External user-provided GTX reference clock on SMA input connectors

• Differential Input

The following figure shows this AC-coupled clock circuit:

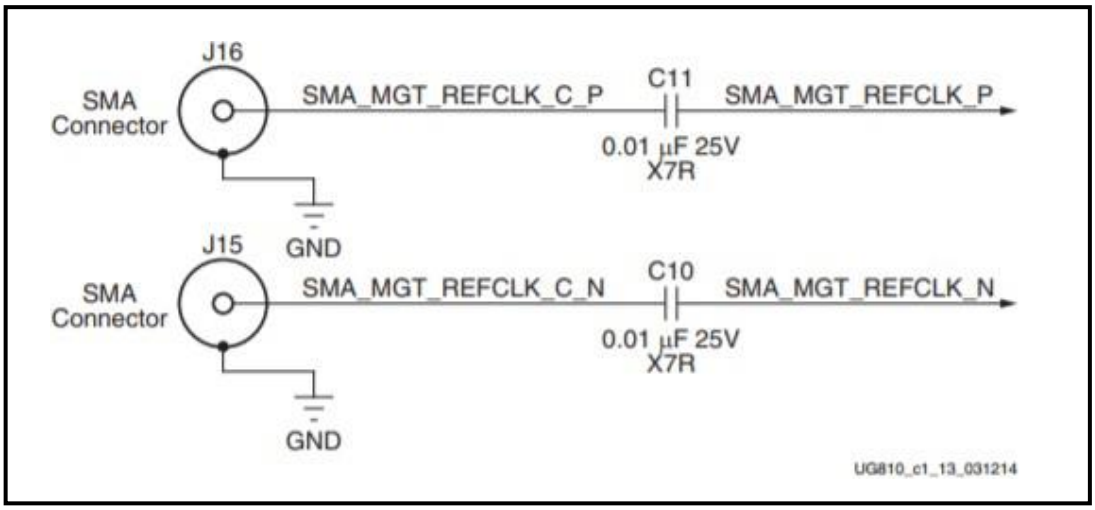

Figure 2.6: GTX SMA Clock Source

Jitter Attenuated Clock

The KC705 board includes a Silicon Labs Si5324 jitter attenuator U70 on the back side of the board. FPGA user logic can implement a clock recovery circuit and then output this clock to a differential I/O pair on I/O bank 13 (REC_CLOCK_C_P, FPGA U1 pin W27 and REC_CLOCK_C_N, FPGA U1 pin W28) for jitter attenuation. The jitter attenuated clock (SI5326_OUT_C_P, SI5326_OUT_C_N) is then routed as a reference clock to GTX Quad 116 inputs MGTREFCLK0P (FPGA U1 pin L8) and MGTREFCLK0N (FPGA U1 pinL7).

The Silicon Labs Si5324 U70 pin 1 reset net SI5326_RST must be driven High to enable the device. U70 pin 1 net SI5326_RST is level-shifted to VADJ by U75 and is connected to U1 bank 12 pin AE20. An active-Low input performs an external hardware reset of the device. This resets all internal logic to a known state

and forces the device registers to their default value. The clock outputs are disabled during reset. The part must be programmed after a reset or power-up to get a clock output. The reset pin 1 has a weak internal pull-up.

The primary purpose of this clock is to support CPRI/OBSAI applications that perform clock recovery from a user-supplied SFP/SFP+ module and use the jitter attenuated recovered clock to drive the reference clock inputs of a GTX transceiver.

2.3.8 GTX Transceivers

The KC705 board provides access to 16 GTX transceivers:

• Eight of the GTX transceivers are wired to the PCI Express® x8 endpoint edge connector (P1)fingers

• Four of the GTX transceivers are wired to the FMC HPC connector(J22)

• One GTX is wired to the FMC LPC connector(J2)

• One GTX is wired to SMA connectors (RX: J17, J18 TX: J19,J20)

• One GTX is wired to the SFP/SFP+ Module connector(P5)

• One GTX is used for the SGMII connection to the Ethernet PHY(U37)

The GTX transceivers in 7 series FPGAs are grouped into four channels described as Quads. The reference clock for a Quad can be sourced from the Quad above or Quad below the GTX Quad of interest. There are four GTX Quads on the KC705 board with connectivity.

2.3.9 PCI Express Edge Connector

PCI Express (Peripheral Component Interconnect Express), officially abbreviated as PCIe or PCI-e, is a high-speed serial computer expansion bus standard, designed to replace the older PCI, PCI-X and AGP bus standards. It is the common motherboard interface for personal computers' graphics cards, hard drives, SSDs, Wi-Fi and Ethernet hardware connections. PCIe has numerous improvements over the older standards, including higher maximum system bus throughput, lower I/O pin count and smaller physical footprint, better performance scaling for bus devices, a more detailed error detection and reporting mechanism (Advanced Error Reporting, AER), and native hot-swap functionality. More recent revisions of the PCIe standard provide hardware support for I/O virtualization.

The 8-lane PCI Express edge connector performs data transfers at the rate of 2.5 GT/s for a Gen1 application and 5.0 GT/s for a Gen2 application. The PCIe transmit and receive signal data paths have a characteristic impedance of 85Ω ±10%. The PCIe clock is routed as a 100Ω differential pair. The 7 series FPGAs GTX transceivers are used for multi-gigabit per second serial interfaces.

The XC7K325T-2FFG900C FPGA (-2 speed grade) included with the KC705 board supports up to Gen2 x8.

The PCIe clock is input from the edge connector. It is AC coupled to the FPGA through the MGTREFCLK1 pins of Quad 115. PCIE_CLK_Q0_P is connected to FPGA U1 pin U8, and the _N net is connected to pin U7. The PCI Express clock circuit is shown below:

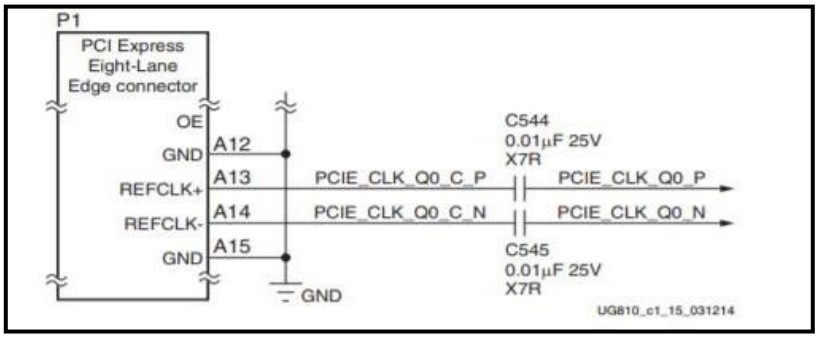

Figure 2.7: PCI Express Clock

PCIe lane width/size is selected via jumper J32 (seen in below figure). The default lane size selection is 8-lane (J32 pins 5 and 6 jumpered).

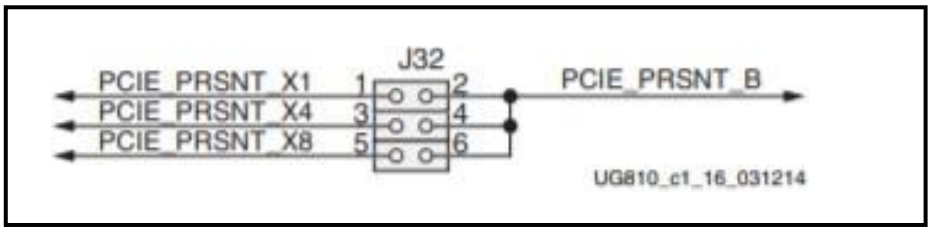

Figure 2.8: PCI Express Lane Size Select Jumper J32

2.3.10SFP/SFP+ Connector:-

The KC705 board contains a small form-factor pluggable (SFP+) connector and cage assembly that accepts SFP or SFP+ modules. The small form-factor pluggable (SFP) is a compact, hot-pluggable network interface module used for both telecommunication and data communications applications. The form factor and electrical interface are specified by a multi-source agreement (MSA) under the

auspices of the Small Form Factor Committee. It is a popular industry format jointly developed and supported by many network component vendors. The advantage of using SFPs instead of fixed interfaces (e.g. modular connectors in Ethernet switches) is that the interface port can be equipped with any suitable type of transceiver as needed.

The SFP+ (enhanced small form-factor pluggable) is an enhanced version of the SFP that supports data rates up to 16 Gbit/s. The SFP+ specification was first published on May 9, 2006, and version 4.1 published on July 6, 2009. SFP+ supports 8 Gbit/s Fibre Channel, 10 Gigabit Ethernet and Optical Transport Network standard OTU2. It is a popular industry format supported by many network component vendors. Although the SFP+ standard does not include mention of 16 Gbit/s Fibre Channel, it can be used at this speed.

2.3.11 10/100/1000 Tri-Speed Ethernet PHY:-

The Tri-Speed Ethernet Media Access Controller (TSMAC) IP core can be configured to operate in either the Gigabit mode (1000Mbits/sec data rate) or the Fast Ethernet mode (10/100 Mbits/sec data rate). Operation in either Gigabit mode or Fast Ethernet mode is selected by setting an internal register bit.

The Tri-Speed Ethernet MAC transmits and receives data between a host processor and an Ethernet network. The main function of the Ethernet MAC is to ensure that the Media Access rules specified in the 802.3 IEEE standard are met while transmitting a frame of data over Ethernet. On the receiving side, the Ethernet MAC extracts the different components of a frame and transfers them to higher applications through the FIFO interface.

The KC705 board utilizes the Marvell Alaska PHY device (88E1111) U37 for Ethernet communications at 10, 100, or 1000 Mb/s. The board supports MII, GMII, RGMII, and SGMII interfaces from the FPGA to the PHY (Table 1-16). The PHY connection to a user-provided Ethernet cable is through a Halo HFJ11-1G01E RJ-45 connector (P3) with built-in magnetics.

Table 2.2: PHY Default Interface Mode

Mode	**Jumper Settings**		
	J29	J30	J64
GMII/MII to copper (default)	Jumper over pins 1-2	Jumper over pins 1-2	No jumper
SGMII to copper, no clock	Jumper over pins 2-3	Jumper over pins 2-3	No jumper
RGMII	Jumper over pins 1-2	No jumper	Jumper on

2.3.12 SGMII GTX Transceiver Clock Generator

An Integrated Circuit Systems ICS844021I chip (U2) generates a high-quality, low-jitter, 125 MHz LVDS clock from a 25 MHz crystal (X3). This clock is sent to FPGA U1, bank 117 GTX transceiver (clock pins G8 (P) and G7 (N)) driving the SGMII interface. Series AC coupling capacitors are present to allow the clock input of the FPGA to set the common mode voltage.

2.3.13 USB-to-UART Bridge

The KC705 board contains a Silicon Labs CP2103GM USB-to-UART bridge device (U12) which allows a connection to a host computer with a USB port. The USB cable is supplied in the Evaluation Kit (standard-A plug to host computer, mini-B plug to KC705 board connector J6). The CP2103GM is powered by the USB 5V provided by the host PC when the USB cable is plugged into the USB port on the KC705 board.

Xilinx UART IP is expected to be implemented in the FPGA fabric. The FPGA supports the USB-to-UART bridge using four signal pins: Transmit(TX), Receive (RX), Request to Send (RTS), and Clear to Send(CTS).

Silicon Labs provides royalty-free Virtual COM Port (VCP) drivers for the host computer. These drivers permit the CP2103GM USB-to-UART bridge to appear as a COM port to communications application software (for example, Tera Term or HyperTerm) that runs on the host computer.

2.3.14 HDMI Video Output

The KC705 board provides a High-Definition Multimedia Interface (HDMI) video output using the Analog Devices ADV7511KSTZ-P HDMI transmitter (U65). The HDMI output is provided on a Molex 500254-1927 HDMI type-A connector (P6). The KC705 board supports the following HDMI device interfaces:

• 18 data lines

• Independent VSYNC, HSYNC

• Single-ended input CLK

• Interrupt Out Pin to FPGA

• I2C

2.3.15 LCD Character Display

A 2-line by 16-character display is provided on the KC705 board.

Figure 2.9: LCD Display

The character display runs at 5.0V and is connected to the FPGA's 1.5V HP bank 33 through a Texas Instruments TXS0108E 8-bit bidirectional voltage level translator (U10). The figure below shows the LCD interface circuit.

2.3.16 I2C Bus Switch

The KC705 board implements a single I2C port on the FPGA, which is routed through a TI PCA9548 1-to-8 channel I2C switch (U49). U49 pin 24 net IIC_MUX_RESET_B is connected to U1 bank 15 pin P23.

This is an active-Low signal and must be driven High (FPGA U1 pin P23) to enable I2C bus transactions between the FPGA U1 and the other components on the I2C bus. The I2C switch can operate at speeds up to 400 kHz.

2.3.17 Status LEDs

The following table defines the status LEDs:

Table 2.3: Status LEDs

Signal Name	Colour	Description
PWRCTL1_VCC4A_PG	Green	FMC power good
FPGA_DONE	Green	FPGA configured successfully
FPGA_INIT_B	Green/red	GREEN: FPGA initialization successful RED: FPGA initialization in progress
VCC12_P_IN	Green	12V power ON
PWRCTL_PWRGOOD	Green	UCD9248 power controllers
LINEAR_POWER_GOOD	Green	TPS51200 (U33) VTTDDR Power Good

2.3.18 Ethernet PHY Status LEDs

The Ethernet PHY status LEDs are mounted to be visible through the metal bracket on the left edge of the KC705 board when it is installed into a slot in a PC chassis. The six PHY status LEDs are located above the RJ45 Ethernet jack as shown in the figure below:

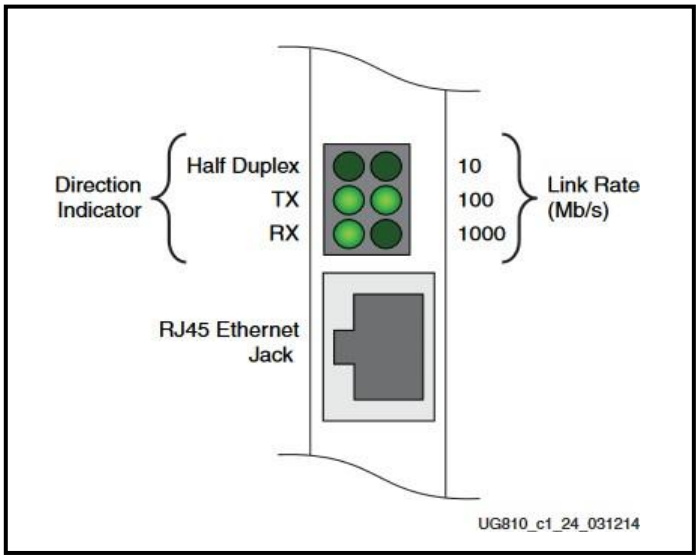

Figure 2.10: Ethernet PHY Status LEDs

The following table lists the Ethernet PHY status LEDs:

Table 2.4: Ethernet PHY Status LEDs

Signal Name	Colour	Description
PHY_LED_RX	GREEN	Ethernet PHY RX
PHY_LED_LINK1000	GREEN	Ethernet Link Speed is 1000 Mb/s
PHY_LED_TX	GREEN	Ethernet PHY TX
PHY_LED_LINK100	GREEN	Ethernet Link Speed is 100 Mb/s
PHY_LED_DUPLEX	GREEN	Ethernet Link is Half-duplex
PHY_LED_LINK10	GREEN	Ethernet Link Speed is 10 Mb/s

2.3.19 User I/O

The KC705 board provides the following user and general purpose I/O capabilities:

• Eight user LEDs

• Five user pushbuttons and reset switch

• 4-position user DIP Switch

• User Rotary Switch

• User SMA

• 2-line x 16-character LCD Character Display

2.3.20 Switches

The KC705 evaluation board includes a power and a configuration switch:

• Power on/off slide switchSW15

• FPGA_PROG_B SW14,active-Low

2.3.21 FPGA Mezzanine Card Interface

FPGA Mezzanine Card (FMC) is an ANSI/VITA 57.1 standard that defines I/O mezzanine modules with connection to an FPGA or other device with re-configurable I/O capability. It specifies a low profile connector and compact board size for compatibility with several industry standard slot card, blade, low profile motherboard, and mezzanine form factors.

Signalling Speed Ratings:

• Single-ended: 9 GHz (18Gb/s)

• Differential Optimal Vertical: 9 GHz (18Gb/s)

• Differential Optimal Horizontal: 16 GHz (32Gb/s)

• High Density Vertical: 7 GHz (15Gb/s)

2.3.22 Power Management

The KC705 board uses power regulators and PMBus compliant digital PWM system controllers from Texas Instruments to supply core and auxiliary voltages. The Texas Instruments Fusion Digital Power graphical user interface (GUI) is used to monitor the current and temperature levels of the board power modules. If any module temperature approaches 85°C, forced air cooling must be provided to keep the module temperature within rated limits.

2.3.23 XADC Header

7 series FPGAs provide an analog-to-digital converter (XADC) block. The XADC block includes a dual 12-bit, 1 MSPS analog-to-digital converter (ADC) and on-chip sensors.

2.4 Ethernet 802.3 Embedded System

2.4.1 Introduction

Ethernet is the most popular wired LAN technology nowadays, and its use is also growing in metropolitan area and wide area networks. Since its introduction in 1980, Ethernet data transfer rates have increased from the original 10Mb/s to the latest 400Gb/s.

Protocols such as Foundation Fieldbus, ControlNet, Interbus, and CAN have been developed specifically for networking embedded real-time systems. These protocols typically run slowly (around 1 Mbit/s) Standard Ethernet as an Embedded Communication Network June 8, 1999 2 but offer increased determinism over multiple access methods used by desktop networks.

Ethernet is a full duplex serial data protocol over twisted pair wires (or wireless). The data may be transmitted to many devices over a network or peer to peer. FTDI FT900 MCU supports full-duplex and half-duplex modes. Auto

negotiation is the procedure by which two connected devices choose common transmission parameters, e.g. speed and duplex mode.

2.4.2 Network Model

OSI Model

The OSI model provides a standard description or "reference model" for how messages should be transmitted between any two points in a telecommunication network. It consists of seven layers and each layer describes the status of the communications.

TCP/IP Model

The TCP/IP model consists of four layers: -

a) Network Layer

The Network layer combines the Data Link layer and Physical layer, including the twisted pair cable, the Physical layer device (PHY), and the Ethernet Media Access Controller (MAC).

b) Internet Layer

The Internet layer consists primarily of a software implementation. The IP header is evaluated or generated by software.

c) Transport Layer

The Transport layer defines what should be done with the data. This layer is based on the following two popular protocols:

(i) UDP is a very simple protocol and is perfect for streaming sequences (e.g., audio or video).

(ii) TCP is a highly reliable host-to-host protocol for a controlled connection. TCP is appropriate for applications that require guaranteed delivery.

d) Application Layer

The Application layer includes all available software implementations (e.g., FTP, HTTP, SMTP, DNS, …) that make up the lower layers. These applications can work only in combination with the API of TCP or UDP, which form the software implementation of the Transport layer.

The figure below illustrates the various layers present in each type of model

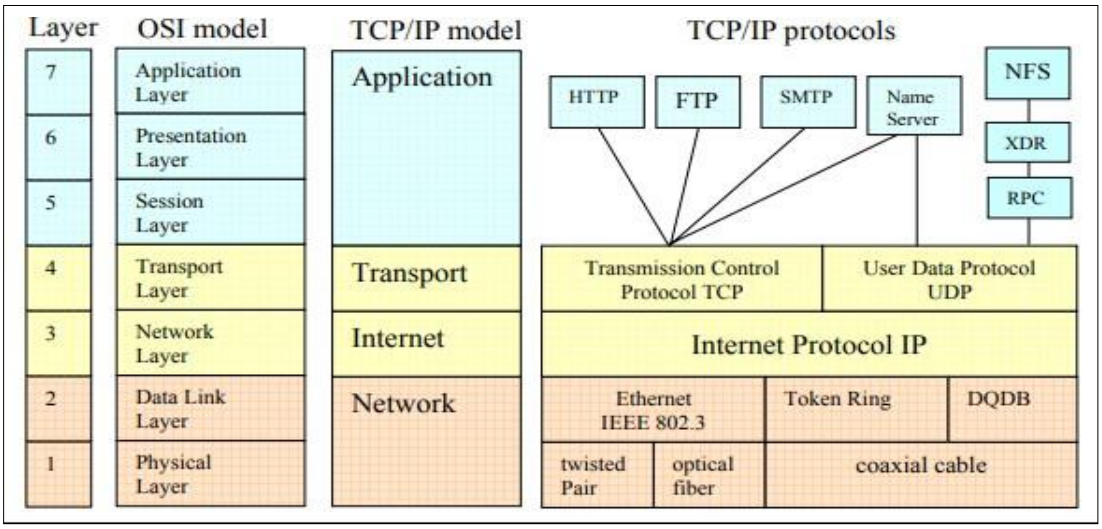

Figure 2.11: Comparison between OSI model AND TCP/IP model

2.4.3 Hardware

The standard connection between the MAC and PHY is the Media Independent Interface (MII). MII Interface is described later in this document after the description of PHY tasks. For 10Mbit/s networks, the PHY works with the Manchester encode and decode mechanism. In 100-Mbit/s networks, the supporting tasks are 4B/5B encode and decode, scrambling, non-return-to zero/inverted (NRZI) coding, and MLT-3 conversion. Auto-Negotiation is based in the AUTONEG level and works all the time.

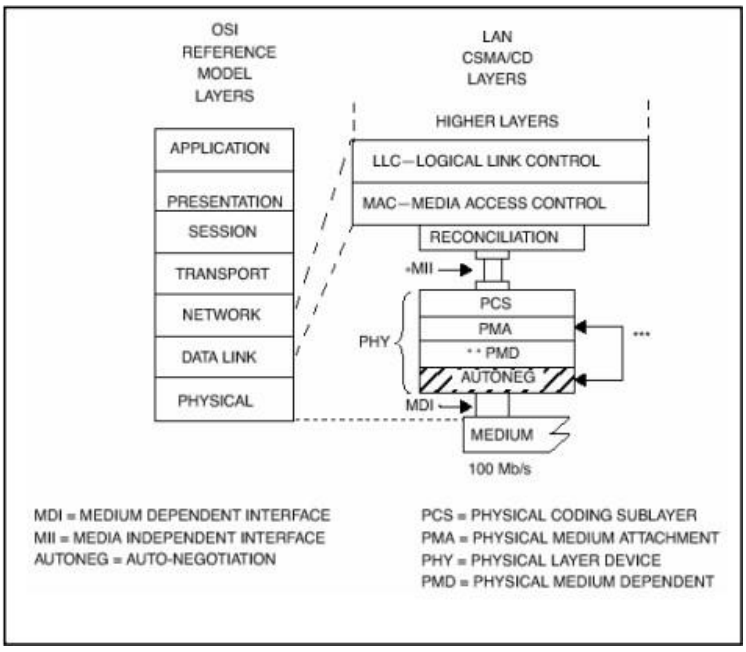

Figure 2.12: PHY and MAC Layer 100-Mbit Network

- MII is optional for 10 Mb/s DTEs and for 100 Mb/s systems and is not specified for 1 Mb/s systems.

- PMD is specified for 100BASE-X only; 100BASE-T4 does not use this layer.
- AUTONEG is optional.

The MAC and PHY blocks are on chip on the FT900

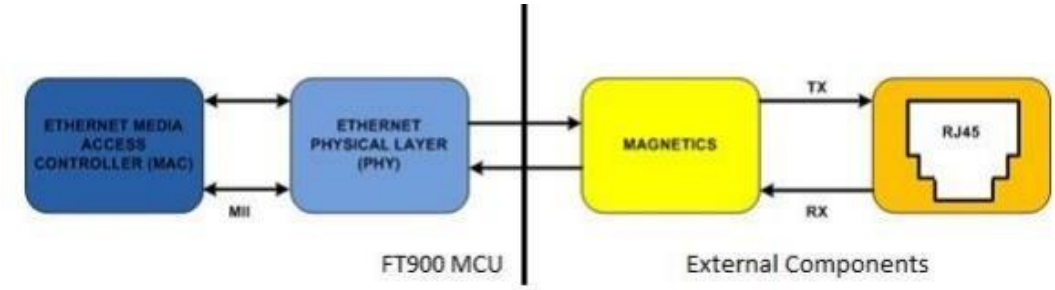

Figure 2.13: Ethernet Link

2.4.4 The Ethernet Media Access Controller (MAC)

The Ethernet media access controller is responsible for managing all data packets that are to be sent and received over a network. The controller is driven by the system software and handles all the addressing and data packetizing. The controller connects directly with the PHY. With the FT900, the link is established via a Media Independent Interface (MII), which is an internal 4-bit wide bi-directional bus (4 transmit data bits, 4 receive data bits)

2.4.5 The Ethernet Physical Layer (PHY)

The Ethernet Physical Layer (PHY) is responsible for the physical link between the Ethernet controller and the network. This layer is on-chip on the FT900 so only the differential signals are available at pin level.

Theoretical Block Diagram of a PHY The MII and PHY signal chain includes some different frequencies and data transformations. Figure 2.14 shows all possible frequencies and all blocks of data changes as well. It is merely a theoretical drawing, but it is representative of the signal chain of a 100Base-TX data stream in general.

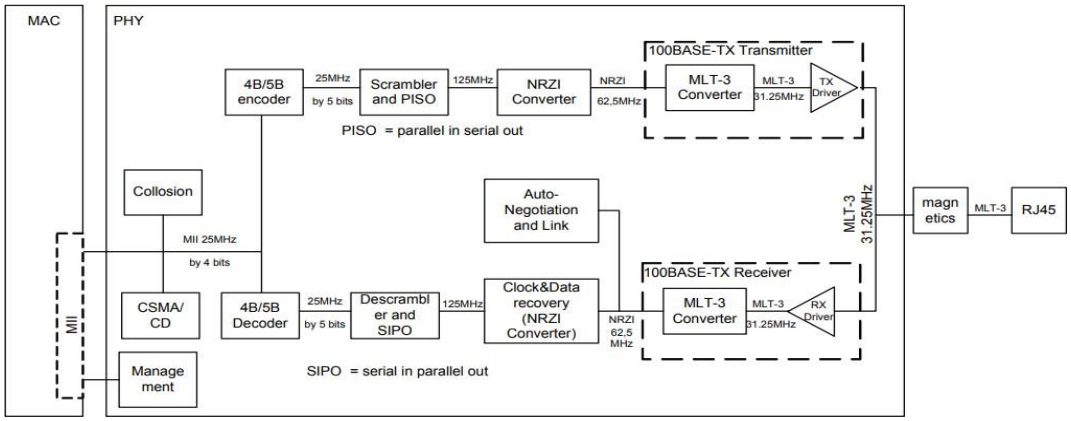

Figure 2.14: Theoretical PHY Block Diagram in 100Base-TX Mode

- Registers of MII compatible PHY- Each PHY has several registers for its configuration. The registers are split into IEEE compliant registers and vendor-specific registers. For instance, the first 16 registers are compliant to IEEE, but it depends on the supported features how many registers are included. This is also valid for the vendor-specific register; this means that the number of the included registers depends on the supported functions of the device.

- The vendor-specific registers start from address 16 until address 32, in general. The registers are accessible via MII Interface using the Station Management pins. It is a synchronous serial interface with a clock (MDC) and a data (MDIO) wire. For more information, refer to MII Interface later in this EE- Note.

- LED -The PHY can indicate the status of the system using LEDs. By setting the appropriate registers, the LEDs will show useful system information, (e.g., indications for Speed-, Link-, Transmit-Receive-, Collision, Duplex Status, and soon).

2.4.6 Addressing

MAC Address

Every Ethernet controller has a unique MAC address to ensure it can be independently identified. The MAC address is 6 bytes in length with the first 3 bytes identifying the Ethernet controller manufacturer. These 3 bytes are controlled and issued by the IEEE Registration Authority. The format of the MAC address is: MM::MM::MM::SS::SS::SS MM identifies the manufacturer.

For example, A0-48-1C is Hewlett Packard. SS identifies a device serial number and is a unique number. There are 224 combinations. The MAC address on FTDI FT900 MCU is programmable.

IP Address

The IP address assigned to a device is used to identify it on a TCP/IP network. As with the MAC address different elements have specific meaning. The basic format is for a 32-bit number split into 4 octets (1 octet = 1 byte) and typically displayed as a series of decimal numbers from 0-255, e.g. 192.168.0.1 is commonly used for accessing Wi-Fi router configuration pages. The first 8 bits define a network class, while the remaining elements define the host within the network.

2.4.7 Half Duplex and Full Duplex Modes of Ethernet

The different modes are described in these IEEE specifications:

- Half-Duplex: 10 MBit/s (IEEE802.3)
- Full-Duplex: 100 MBit/s (IEEE802.3u)
- Full-Duplex: Gigabit-Ethernet (IEEE 802.3ab) is Full-Duplex transfer with all four pairs.

2.4.8 MII Interface

The media-independent interface (MII) was originally defined as a standard interface to connect a Fast Ethernet (i.e., 100 Mbit/s) media access control (MAC) block to a PHY chip. The MII is standardized by IEEE 802.3u and connects different types of PHYs to MACs. Being media independent means that different types of PHY devices for connecting to different media (i.e. twisted pair, fiber optic, etc.) can be used without redesigning or replacing the MAC hardware. Thus, any MAC may be used with any PHY, independent of the network signal transmission media.

The Media Independent Interface (MII) is an 18- pin interface and is designed for communications between PHY and MAC layers in 10Mbit and 100Mbit Ethernet networks. The transceiver (PHY) can be connected directly or with cables and connectors (maximum distance is of 50 cm).

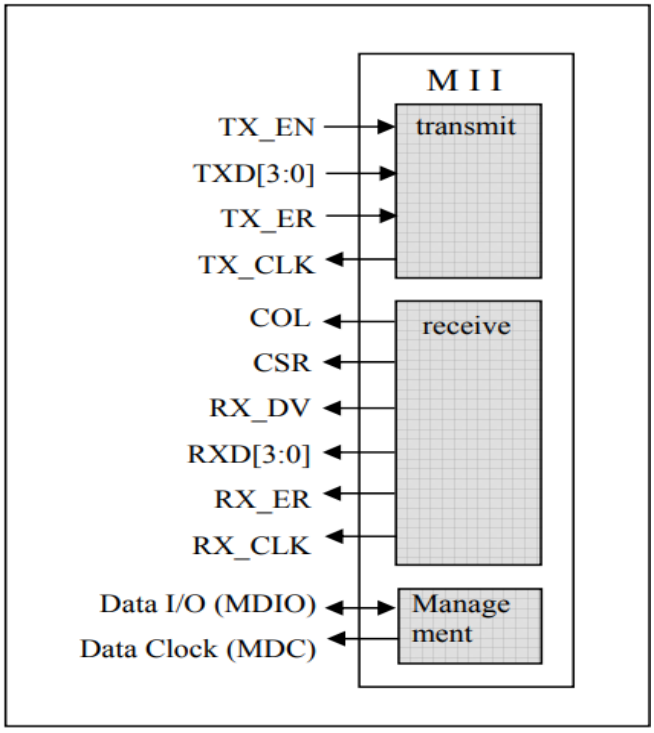

Figure 2.15: Description of MII

2.4.9 MDIO Interface

Management Data Input/Output (MDIO), also known as Serial Management Interface (SMI) or Media Independent Interface Management (MIIM), is a serial bus defined for the Ethernet family of IEEE 802.3 standards for the Media Independent Interface, or MII. The MII connects Media Access Control(MAC)devices with Ethernet physical layer (PHY) circuits. The MAC device controlling the MDIO is called the Station Management Entity (SME).

Relationship with MII

MII has two signal interfaces:

- A Data interface to the Ethernet MAC, for sending and receiving Ethernet frame data.
- A PHY management interface, MDIO, used to read and write the control and status registers of the PHY in order to configure each PHY before operation, and to monitor link status during operation.

Electrical Specifications

The MDIO interface is implemented by two signals:

- **MDC clock:** driven by the MAC device to the PHY.
- **MDIO data:** bidirectional, the PHY drives it to provide register data at the end of a read operation.

2.4.10 Ethernet Frame

A data packet on an Ethernet link transports an Ethernet frame. The internal structure of an Ethernet frame is specified in IEEE 802.3-2012.

Layer	Preamble	Start of Frame Delimiter	MAC Dest.	MAC Source	802.1Q Tag (optional)	Length	Payload	Frame Check (32 bit CRC)	Interpacket Gap
	7 octets	1 octet	6 octets	6 octets	4 octets	2 octets	1500 octets max	4 octets	12 octets
Layer 2 Frame									
Layer 1 Packet									

Figure 2.16: Ethernet Frame Structure

1) Preamble: Seven octets of 55h. In 100 Mb/s operation, the first octet is 4B/5B encoded to /J/K/ (more on what this means later), and is known as the Start-of-

Stream Delimiter (SSD). The preamble is present to allow the receiver to lock onto the stream of data before the actual frame arrives.

2) **Start-of-Frame Delimiter (SFD):** The SFD is sometimes considered to be part of the preamble. This is why the preamble is sometimes described as eight octets.

3) **Destination MAC Address:** The 6-octet MAC address of the destination hardware.

4) **Source MAC Address:** The 6-octet MAC address of the source hardware.

5) **Length/Type:** If the value in this 2-octet field is ≤ 1500 (decimal), this represents the number of octets in the payload. If the value is ≥ 1536, this represents the EtherType (payloadtype).

6) **Payload (Client Data):** The client data, such as an IP datagram, etc. The minimum payload size is 46 octets; the maximum payload size is 1500 octets. While payloads below or above these limits do not meet the IEEE 802.3 specification, there is varied support for these payloads depending on the particular vendor.

7) **Pad:** Since the minimum payload size is 46 octets, pad octets must be inserted to reach this minimum if the payload size is less than 46octets.

8) **Frame Check Sequence (FCS):** The value of the 4-octet FCS field is calculated over the source address, destination address, length/type, data and pad fields using a 32-bit Cyclic Redundancy Check(CRC).

9) **End-of-Stream Delimiter(ESD):** In 100Mb/s operation, the PHY transmits a /T/R/ symbol pair after the FCS (during the inter-frame gap) to denote the end of the frame. In 10 Mb/s operation, a special TP_IDL signal and network silence indicates the end of the frame. This TP_IDL marker is not considered part of frame data.

2.5 UART

2.5.1 Introduction

The Universal Asynchronous Receiver Transmitter (UART) is a popular and widely-used device for data communication in the field of telecommunication. There are different versions of UARTs in the industry. Some of them contain FIFOs for the receiver/transmitter data buffering and some of them have the 9 data bits mode (start bit +9 data bits + parity + stop bits).

Figure 2.17 shows the major blocks implemented in the design. The receiver performs serial-to-parallel conversion on the asynchronous data frame received from the serial data input SIN. The transmitter performs parallel-to-serial conversion on the

8-bit data received from the CPU. In order to synchronize the asynchronous serial data and to ensure the data integrity, start, parity and stop bits are added to the serial data.

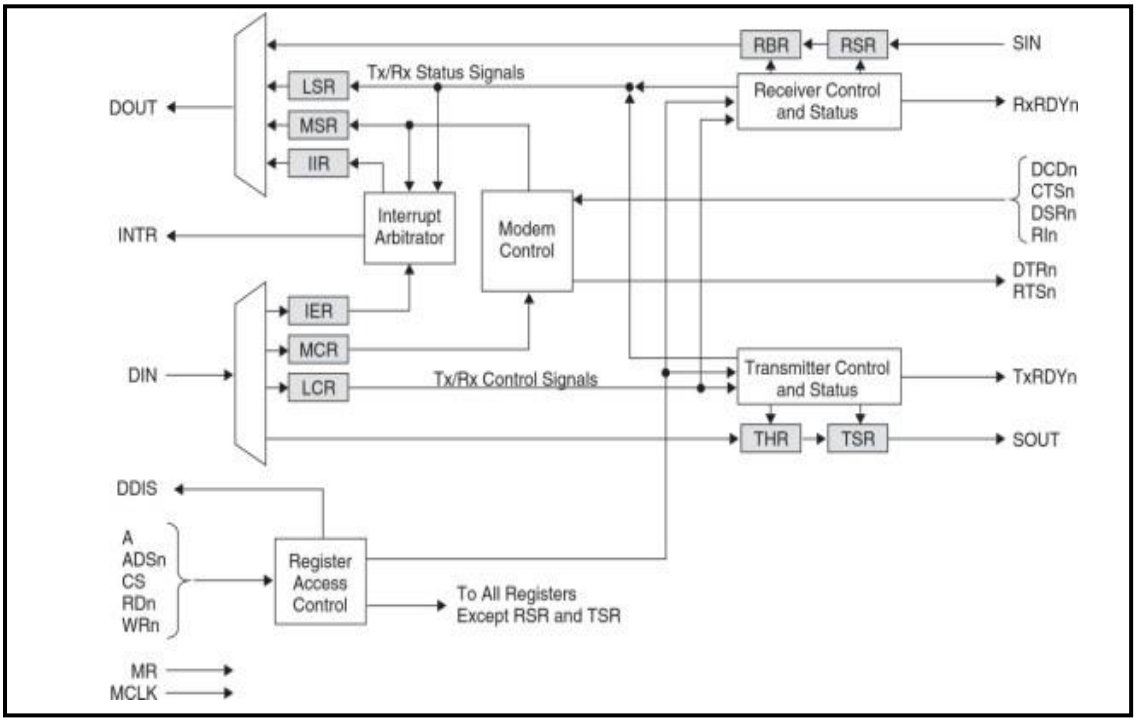

Figure 2.17: UART Block Diagram

A UART (Universal Asynchronous Receiver/Transmitter) is the microchip with programming that controls a computer's interface to its attached serial devices. Specifically, it provides the computer with the RS-232C Data Terminal Equipment (DTE) interface so that it can "talk" to and exchange data with modems and other serial devices. As part of this interface, the UART also:

- Converts the bytes it receives from the computer along parallel circuits into a single serial bit stream for out bound transmission
- On in bound transmission, converts the serial bit stream into the bytes that the computer handles
- Adds a parity bit (if it's been selected) on outbound transmissions and checks the parity of incoming bytes (if selected) and discards the parity bit
- Adds start and stop delineators on outbound and strips them from inbound transmissions
- Handles interrupts from the keyboard and mouse (which are serial devices with special ports)
- May handle other kinds of interrupt and device management that require coordinating the computer's speed of operation with device speeds.

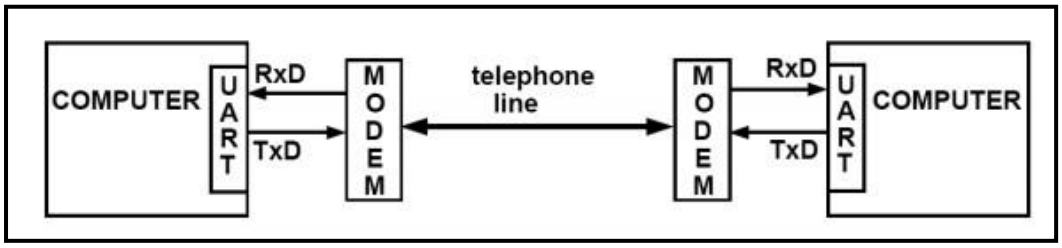

Fig 2.18: Serial Transmission

The communications links across which computers—or parts of computers—talk to one another may be either serial or parallel. A parallel link transmits several streams of data along multiple channels (wires, printed circuit tracks, optical fibres, etc.); a serial link transmits a single stream of data.

A number of factors allow serial to be clocked at a greater rate:

- Clock skew between different channels is not an issue
- A serial connection requires fewer interconnecting cables (e.g. wires/fibres) and hence occupies less space. The extra space allows for better isolation of the channel from its surroundings
- Crosstalk is less of an issue, because there are fewer conductors in proximity.

2.5.2 Synchronous Serial Transmission

Synchronous serial transmission requires that the sender and receiver share a clock with one another, or that the sender provide a strobe or other timing signal so that the receiver knows when to “read” the next bit of the data. In most forms of serial Synchronous communication, if there is no data available at a given instant to transmit, a fill character must be sent instead so that data is always being transmitted. Synchronous communication is usually more efficient because only data bits are transmitted between sender and receiver, and synchronous communication can be more costly if extra wiring and circuits are required to share a clock signal between the sender and receiver. A form of Synchronous transmission is used with printers and fixed disk devices in that the data is sent on one set of wires while a clock or strobe is sent on a different wire. Printers and fixed disk devices are not normally serial devices because most fixed disk interface standards send an entire word of data for each clock or strobe signal by using a separate wire for each bit of the word. In the PC industry, these are known as Parallel devices. The standard serial communications hardware in the PC does not support Synchronous operations. This mode is described here for comparison purposes only.

2.5.3 Asynchronous Serial Transmission

Asynchronous serial communication describes an asynchronous transmission protocol in which a start signal is sent prior to each byte, character or code word and a stop signal is sent after each code word. The start signal serves to prepare the receiving mechanism for the reception and registration of a symbol and the stop signal serves to bring the receiving mechanism to rest in preparation for the reception of the next symbol. A common kind of start-stop transmission is ASCII over RS-232, for example for use in tele-typewriter operation.

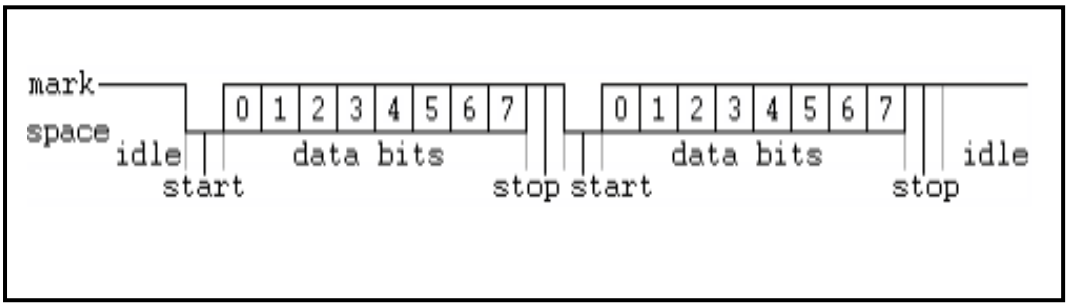

Figure 2.19: Asynchronous Serial Transmission

In the diagram, a start bit is sent, followed by eight data bits, no parity bit and one stop bit, for a 10-bit character frame. The number of data and formatting bits, and the transmission speed, must be pre-agreed by the communicating parties. After the stop bit, the line may remain idle indefinitely, or another character may immediately be started.

2.5.4 Bits, Baud and Symbols

Baud is a measurement of transmission speed in asynchronous communication. Traditionally, a Baud Rate represents the number of bits that are actually being sent over the media.

The Baud count includes the overhead bits Start, Stop and Parity that are generated by the sending UART and removed by the receiving UART. This means that seven-bit words of data actually take 10 bits to be completely transmitted. Therefore, a modem capable of moving 300 bits per second from one place to another can normally only move 30 7-bit words if Parity is used and one Start and Stop bit are present. Modern high-speed modems (2400, 9600, 14,400, and 19,200bps) in reality still operate at or below 2400 baud, or more accurately, 2400 Symbols per second.

2.5.5 Asynchronous Serial Reception

```
                 start|<-  five to eight data bits  ->| stop bit(s)
             0    ----  -  -  -  -  -  -  -  -  -  -  -
Space(logic low, high data-wire voltage)
                  |   |   |   |   |   |   |   |   |   |   |   |
                  | S | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | S | S |
                  |   |   |   |   |   |   |   |   |   |   |   |
             1        -  -  -  -  -  -  -  -  -  -  -  --------
Mark(logic high, low data-wire voltage)
```

Figure 2.20: Asynchronous serial reception

A multiplexed data communication pulse can only be in one of two states but there are many names for the two states. When on, circuit closed, low voltage, current flowing, or a logical zero, the pulse is said to be in the "space" condition. When off, circuit open, high voltage, current stopped, or a logical one, the pulse is said to be in the "mark" condition. A character code begins with the data communication circuit in the space condition.

2.6 Conclusion

This chapter discusses the dominant hardware which were used in this UART to Ethernet converter, i.e., the Kintex-7 KC705 Evaluation Board, the IEEE 802.3 Ethernet, and the UART.

The Kintex-7 FPGA KC705 evaluation board includes all the basic components of hardware, design tools, IP, and pre-verified reference designs including a targeted design enabling high-performance serial connectivity and advanced memory interfacing. Ethernet is a family of computer networking technologies commonly used in local area networks, metropolitan area networks and wide area networks. It was commercially introduced in 1980 and first standardized in 1983 as IEEE 802.3. A universal asynchronous receiver-transmitter is a computer hardware device for asynchronous serial communication in which the data format and transmission speeds are configurable. The electric signaling levels and methods are handled by a driver circuit external to the UART.

Chapter 3

IP Cores

3.1 Introduction

The circuit design of the UART to Ethernet converter requires and embodies several IP cores. The central ones, among the many, are the MicroBlaze (Soft) Processor, the BRAM controller, and obviously, the UART and the Ethernet. This chapter aims to list out all the components (i.e. the IP cores) present in the circuit along with a detailed explanation about each of them.

3.2 IP Cores

In electronic design a semiconductor intellectual property core, IP core, or IP block is a reusable unit of logic, cell, or integrated circuit (commonly called a "chip") layout design that is the intellectual property of one party. Intellectual Property (IP) refers to preconfigured logic functions that can be used in our design. Xilinx provides a wide selection of IP that is optimized for Xilinx FPGAs. Xilinx produces IP ranging in complexity from simple arithmetic operators and delay elements to complex system-level building blocks, such as Digital Signal Processing (DSP) filters, multiplexers, transformers, and memory.

IP cores fall into one of three categories: hard cores , firm cores , or soft cores . Hard cores are physical manifestations of the IP design. These are best for plug-and-play applications, and are less portable and flexible than the other two types of cores. Like the hard cores, firm (sometimes called semi-hard) cores also carry placement data but are configurable to various applications. The most flexible of the three, soft cores exist either as a net list (a list of the logic gate s and associated interconnections making up an integrated circuit) or hardware description language (HDL) code.

3.3 List of IP cores used

The following table lists all the IP cores which have used to design the UART to Ethernet converter, along with a brief description about each: -

Table 3.1 IP Cores

Name of the IP Core	Description
MicroBlaze Processor	Soft microprocessor core designed for Xilinx field-programmable gate arrays.
MicroBlaze Debug Module (MDM)	Enables JTAG-based debugging of one or more MicroBlaze processors
Processor System Reset	Handles the reset conditions for a given system
AXI Interconnect	Allows any mixture of AXI master and slave devices to be connected to it
AXI Interrupt Controller	Concentrates multiple interrupt inputs from peripheral devices to a single interrupt output to the system processor
AXI UART Lite	Provides a controller interface for asynchronous serial data transfer
AXI Ethernet Lite	Provides a 10 Mb/s and 100 Mb/s (also known as Fast Ethernet) interface
AXI BRAM Controller	AXI Endpoint slave IP for integration with the AXI interconnect
Block Memory Generator	Provides convenient access to memories for a wide range of configurations
Clocking Wizard	Delivers a timing parameter
Concat	Concatenating bus signals of varying widths

3.4 MicroBlaze Processor

3.4.1 Introduction

The MicroBlaze is a soft microprocessor core designed for Xilinx field-programmable gate arrays (FPGA). As a soft-core processor, MicroBlaze is implemented entirely in the general-purpose memory and logic fabric of Xilinx FPGAs. The MicroBlaze has a versatile interconnect system to support a variety of embedded applications. MicroBlaze's primary I/O bus, the AXI interconnect, is a system-memory mapped transaction bus with master–slave capability.

3.4.2 Architecture

The MicroBlaze embedded processor soft core is a reduced instruction set computer (RISC). RISC (Reduced Instruction Set Computer) is used in portable devices due to its power efficiency. Figure 3.1 shows a functional block diagram of the MicroBlaze core.

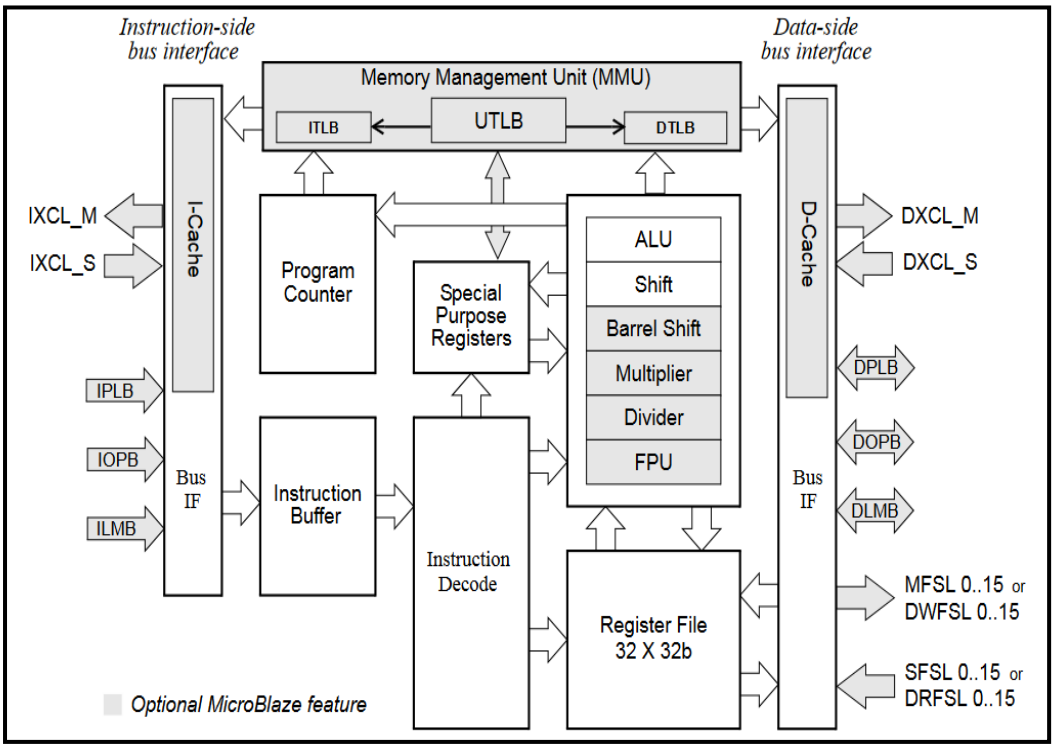

Figure 3.1: MicroBlaze Core Diagram

MicroBlaze is implemented with a Harvard memory architecture; instruction and data accesses are done in separate address spaces. Each address space has a 32-bit range (that is, handles up to 4-GB of instructions and data memory respectively). The instruction and data memory ranges can be made to overlap by mapping them both to the same physical memory. The latter is useful for software debugging.

Both instruction and data interfaces of MicroBlaze are 32 bits wide and use big-endian, bit-reversed format. MicroBlaze supports word, half word, and byte accesses to data memory. The MicroBlaze instruction and data caches can be configured to use 4- or 8-word cache lines. When using a longer cache line, more bytes are pre-fetched.

3.4.3 Features

The MicroBlaze soft core processor is highly configurable, allowing us to select a specific set of features required by our design. The fixed feature set of the processor includes:

•Thirty-two 32-bit general purpose registers

•32-bit instruction word with three operands and two addressing modes

•32-bit address bus

•Single issue pipeline

In addition to these fixed features, the MicroBlaze processor is parameterized to allow selective enabling of additional functionality.

3.4.4 Instructions

All MicroBlaze instructions are 32 bits and are defined as either Type A or Type B. Type A instructions have up to two source register operands and one destination register operand. Type B instructions have one source register and 16-bit immediate operand Type B instructions have a single destination register operand. Instructions are provided in the following functional categories: arithmetic, logical, branch, load/store, and special.

3.4.5 Registers

MicroBlaze has an orthogonal instruction set architecture. It has thirty-two 32-bit general purpose registers and up to eighteen 32-bit special purpose registers, depending on the configured options.

General Purpose Registers

The thirty-two 32-bit General Purpose Registers are numbered R0 through R31. The register file is reset on bit stream (reset value is 0x00000000).The register file is not reset by the external reset inputs: Reset and Debug_Rst, and can only be done using the bit reset value mentioned previously.

Special Purpose Register

The MicroBlaze Processor is equipped with eighteen special purpose registers. Each of these is of 32 bits. Some special purpose registers are as follows: -

(a) Program Counter: - The Program Counter (PC) is the 32-bit address of the

execution instruction. It can be read with an MFS instruction, but it cannot be written with an MTS instruction.

(b) Machine Status Register (MSR): - The Machine Status Register contains control and status bits for the processor. It can be read with an MFS instruction. When reading the MSR, bit 29 is replicated in bit 0 as the carry copy. MSR can be written using either an MTS instruction.

(c) Exception Address Register (EAR): - The Exception Address Register stores the full load/store address that causes the exception. These exceptions are pre-defined. The contents of this register are undefined for all other exceptions.

(d) Exception Status Register (ESR): - The Exception Status Register contains status bits for the processor.

(e) Floating Point Status Register (FSR): - The Floating-Point Status Register contains status bits for the floating-point unit. It can be read with an MFS, and written with an MTS instruction.

3.4.6 Pipeline

MicroBlaze instruction execution is pipelined. For most instructions, each stage takes one clock cycle to complete. Consequently, the number of clock cycles necessary for a specific instruction to complete is equal to the number of pipeline stages, and one instruction is completed on every cycle.

For a 3-stage pipeline, when area optimization is enabled, the pipeline is divided into three stages to minimize hardware cost: Fetch, Decode, and Execute. For a 5-stage pipeline, when area optimization is disabled, the pipeline is divided into five stages to maximize performance: Fetch (IF), Decode (OF), Execute (EX), Access Memory (MEM), and lastly, Writeback (WB).

3.5 MicroBlaze Debug Module(MDM)

MicroBlaze Debug Module (MDM) core enables JTAG-based debugging of one or more MicroBlaze processors. The MDM core is connected to the MicroBlaze processors to be debugged. It supports for debugging up to 32 MicroBlaze processors. It also supports for synchronized control of multiple MicroBlaze processors. It includes a cross-trigger capability, which enables routing of trigger events between connected MicroBlaze processors, as well as an external interface compatible with the

Zynq Processing System. The supports MicroBlaze parallel debug access, designed to provide faster direct access to MicroBlaze debug registers, and to improve timing compared to serial debug.

The MDM includes a UART with a configurable slave bus interface which can be configured for an AXI4-Lite interconnect. The UART TX and RX signals are transmitted over the device JTAG port. In general, it is recommended to only use this feature when software debug through JTAG is not required, because otherwise the MDM must perform a serial to parallel conversion of the JTAG signals, which requires additional logic.

3.6 Processor System Reset

The Processing System Reset is a soft IP that provides a mechanism to handle the reset conditions for a given system. The core handles numerous reset conditions at the input and generates appropriate resets at the output. This core generates the resets based upon external or internal reset conditions.

3.6.1 Features

•Supports asynchronous external reset input which is synchronized with clock

•Both external and auxiliary reset inputs are selectable as active-High or active-Low

•Parameterized Active-Low reset signal generation for core and for interconnect

•Sequencing of reset signals coming out of reset:

a. Bus structures come out of reset (Interconnect and bridge)

b. Peripherals come out of reset 16 clock cycles later (UART, SPI,IIC)

c. The MicroBlaze processor comes out of reset 16 clock cycles after the peripherals

3.6.2 Module Circuit Description

The core has five input signals and five output signals. We can set eight generics. Additional outputs can be generated through the use of the generics, No. of Bus Reset (active-High), No. of Peripheral Reset (active-High), No. of Interconnect Reset (active-Low), and No. of Peripheral Reset (active-Low). The reset signal polarity for peripherals connected to interconnects are active-Low when asserted.

The core should be connected to the slowest operating clock of the system.

The external reset input port should be connected to the external reset pin, while the auxiliary reset input should be connected to the internal source for reset. The MicroBlaze Debug Reset should be connected to the debug module of the system. In the case where the auxiliary reset input and/or MicroBlaze Debug Reset ports are left un-connected, then, these ports are driven with the de-asserted values.

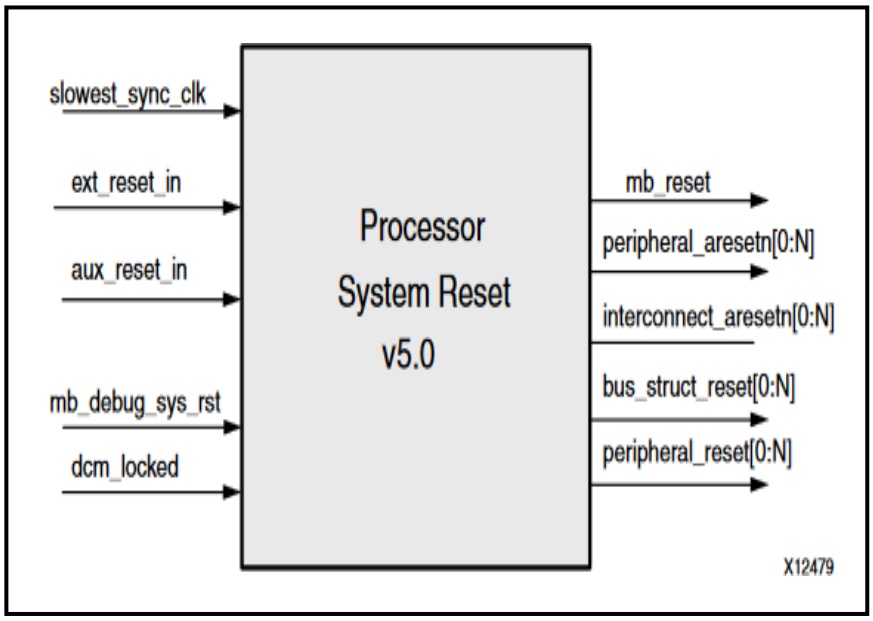

Figure 3.2: Block Diagram of Processor System Reset Module

3.7 AXI Interconnect

The AXI Interconnect core allows any mixture of AXI master and slave devices to be connected to it, which can vary from one another in terms of data width, clock domain and AXI sub-protocol (AXI4, AXI3, or AXI4-Lite). When the interface characteristics of any connected master or slave device differ from those of the crossbar switch inside the interconnect, the appropriate infrastructure cores are automatically inferred and connected within the interconnect to perform the necessary conversions.

Features

• Address width: Up to 64bits

• USER width (per channel): Up to 1,024 bits

• ID width: Up to 32bits

• Support for Read: only and Write-only masters and slaves, resulting in reduced resource utilization

3.8 AXI Interrupt Controller

Interrupt Controller (INTC) core receives multiple interrupt inputs from peripheral devices and merges them into an interrupt output to the system processor. The registers used for storing interrupt vector addresses, checking, enabling and acknowledging interrupts are accessed through the AXI4-Lite interface.

3.8.1 Features

- Fast Interrupt mode
- Supports up to 32 interrupts which are cascadable to provide additional interrupt inputs
- Bus or single interrupt output
- Priority between interrupt requests is determined by vector position. The least significant bit (LSB, in this case bit 0) has the highest priority
- Interrupt Enable Register for selectively enabling individual interrupt inputs.
- Master Enable Register for enabling interrupts request output
- Each input is configurable for edge or level sensitivity
- Support for nested Interrupts

3.8.2 Blocks

The three main blocks in the AXI INTC core are:

- Registers Block: This block contains control and status registers. They are accessed through the AXI4-lite slave interface.
- Interrupt Detection: This block detects the interrupts input. It can be configured for either level or edge detection for each interrupt input.
- Interrupt Generation: This block performs the following functions:
  (a) Generates the final output interrupt from the interrupt controller core.
  (b) Interrupt sensitivity is determined by the configuration parameters.
  (c) Checks for enable conditions in control registers (MER and IER) for interrupt generation.
  (d) Resets the interrupt after acknowledge.

3.9 AXI UART Lite

The LogiCORE IP AXI Universal Asynchronous Receiver Transmitter (UART) Lite core provides between UART signals and the Advanced Microcontroller Bus Architecture (AMBA) AXI interface and also provides a controller interface for asynchronous serial data transfer. This soft LogiCORE IP core is designed to interface with the AXI4-Lite protocol.

3.9.1 Block Diagram

The AXI UART Lite modules are shown in Figure 3.3 and described in the sections that follow.

AXI Interface: This block implements the AXI4-Lite slave interface for register access and data transfer.

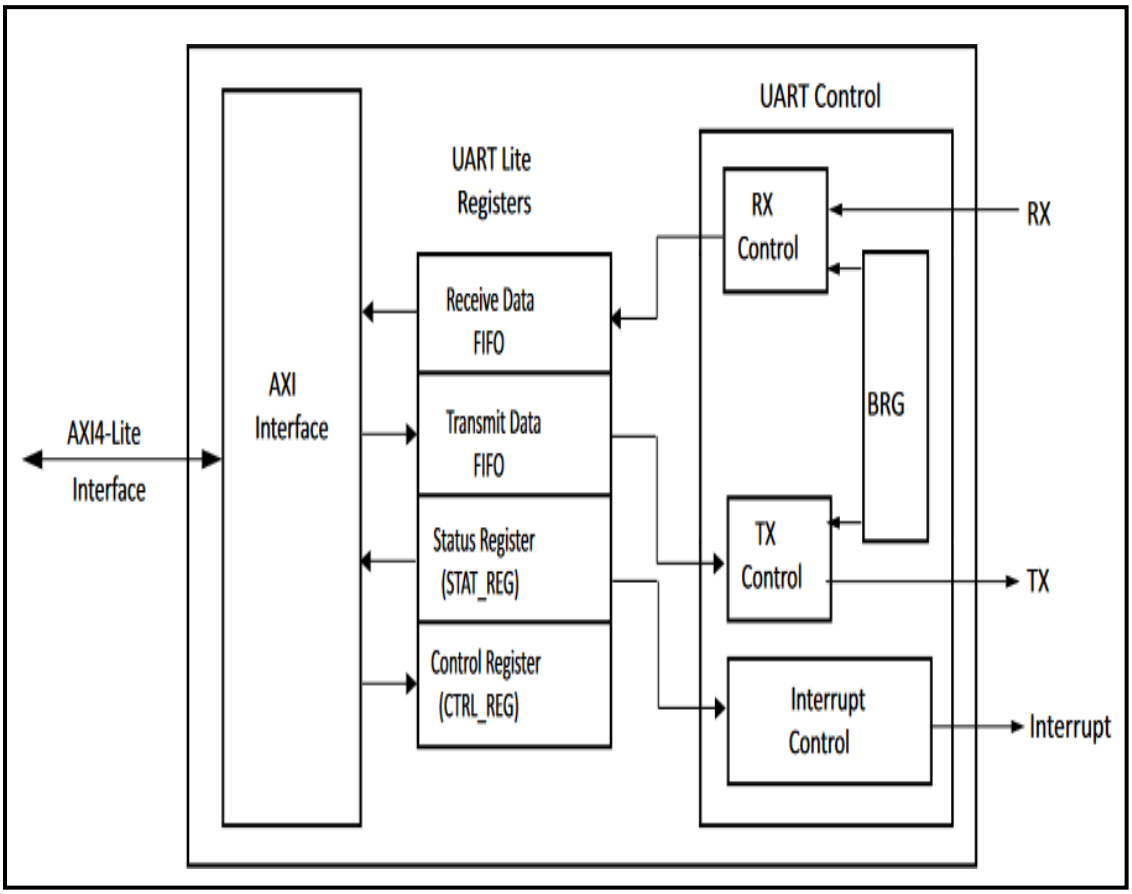

Figure 3.3: Block Diagram of AXI UART Lite

UART Lite Registers: This block includes memory mapped registers (as shown in Figure 3.3). It consists of a control register, a status register, and a pair of transmit/receive FIFOs, both of 16-character depth.

UART Control: This block consists of:

- **Tx Control:** This block reads data from Transmit Data FIFO and sends it out on the UART Tx interface.
- **Rx Control:** This block samples received data with respect to generated baud rate and writes it to Receive Data FIFO.
- **BRG (Baud Rate Generator):** This block generates various baud rates when programmed by the user.
- **Interrupt Control:** The AXI UART Lite core provides interrupt enable/disable control. If interrupts are enabled, a rising-edge sensitive interrupt is generated when the receive FIFO becomes non-empty or when the transmit FIFO becomes empty.

3.9.2 Features

The AXI UART Lite has the following features: -

- Performs parallel-to-serial conversion on characters received through the AXI4-Lite interface and serial-to-parallel conversion on characters received from a serial peripheral.
- Transmits and receives 8, 7, 6, or 5-bit characters, with one stop bit and with odd, even, or no parity bit. The AXI UART Lite can transmit and receive independently.
- Generates a rising-edge sensitive interrupt when the receive FIFO becomes non- empty or when the transmit FIFO becomes empty. This interrupt can be masked by using an interrupt enable/disable signal.
- The device contains a baud rate generator and independent 16-character deep transmit and receive FIFOs.

3.10 AXI Ethernet Lite

AXI Ethernet Lite Media Access Controller (MAC) core is designed to incorporate the applicable features which are described in the IEEE Std. 802.3 Media Independent Interface (MII) specification. It communicates with the processor using

the AXI4 or AXI4-Lite interface. The design provides a 10 Mb/s and 100 Mb/s (also known as Fast Ethernet) interface.

3.10.1 Transmitter

This block consists of transmit logic, Cyclic Redundancy Check (CRC) generator module, transmit data MUX, TX First-In First-Out (FIFO) and the transmit interface module. The CRC generator module calculates the CRC for the frame to be transmitted. The transmit control MUX arranges this frame and sends the preamble, Start-of-Frame Delimiter (SFD), frame data, padding and CRC to the transmit FIFO in the required order. When the frame is transmitted to the PHY, this module generates a transmit interrupt and updates the transmit control register.

3.10.2 Receiver

This block consists of the RX interface, loopback control MUX, RX FIFO, CRC checker and Receive Control module. Receive data signals from the PHY are passed through the loopback control MUX and then stored in the RX FIFO. If loopback is enabled, data on the TX lines is passed to the RX FIFO. The CRC checker module calculates the CRC of the received frame and if the correct CRC is found, the receive control logic will generate the frame receive interrupt.

3.10.3 Features

- Parameterized AXI4 slave interface based on the AXI4 or AXI4-Lite specification for transmit and receive data dual port memory access
- Media Independent Interface (MII) for connection to external 10/100 Mb/s PHY transceivers
- Independent internal 2K byte TX and RX dual port memory for holding data for one packet
- Optional dual buffer memories, 4K byte ping-pong, for TX and RX
- Receive and Transmit Interrupts
- Optional Management Data Input/Output (MDIO) interface for PHY access
- Internal loop back support
- Accepts messages addressed to its uni-cast address and the broad cast address.

3.11 AXI BRAM Controller

The core is designed as an AXI Endpoint slave IP for integration with the AXI interconnect and system master devices to communicate to local block RAM. The core supports both single and burst transactions to the block RAM and is optimized for performance. The AXI BRAM Controller core can be configured such that a single port to the BRAM block or both ports to the BRAM block are utilized in either an AXI4 or AXI4-Lite controller configuration.

3.11.1 BRAM Interface

The BRAM interface is optimized to provide the highest performance interface to the FPGA BRAM module. Port A of the BRAM module is designated as the write port, while Port B of the BRAM module is designated as the read port.

For decreased size, the AXI BRAM Controller can be configured in a single port utilization to the BRAM module. In this case, Port A is used for both read and write operations to block RAM. To reduce resource utilization and limit any potential impacts to latency, any collisions between the read and write ports of the block RAM are not detected by the AXI BRAM Controller.

3.11.2 Supported Memory Size

The AXI BRAM Controller supports memory sizes up to a maximum of 2 MBytes (byte size 8 or 9).The AXI BRAM Controller IP supports a minimum depth of 512 bytes. Any depth that is less than 512 gets reset to 512bytes.

3.11.3 Features

- AXI4 (memory mapped) slave interface
- Low latency memory controller
- Separate read and write channel interfaces to utilize dual port FPGA BRAM technology
- Option to arbitrate read and write data for use with a single port of block RAM (in AXI4 and AXI4LITEmodes)

- Configurable BRAM data width (32-, 64-, 128-, 256-, 512-, and1024-bit)
- Supports memory sizes up to a maximum of 2 MBytes (byte size 8 or9
- Compatible with Xilinx AXI Interconnect
- Supports two-deep address pipelining on each read and write channel (order must be maintained)
- ECC is only supported for BRAM data widths of 32-, 64-, or 128-bit configurations.

3.11.4 Application

The AXI BRAM Controller core is designed to integrate in an AXI system via the AXI Interconnect topology to provide multiple masters access to block RAM. The AXI BRAM Controller is an endpoint slave IP core to be attached as a slave AXI device on the AXI Interconnect. The AXI Interconnect allows the AXI BRAM Controller to be optimized for latency and reduce the resources utilized. The AXI Interconnect performs any mismatching of master data widths to the size of the block RAM and the AXI BRAM Controller core.

3.12 Block Memory Generator

The Block Memory Generator core is used to extend the functionality and capability of a single primitive to memories of arbitrary widths and depths. Sophisticated algorithms within the Block Memory Generator core produce optimized solutions to provide convenient access to memories for a wide range of configurations.

This core has two fully independent ports that access a shared memory space. Both A and B ports have a write and a read interface. This core is not completely backward-compatible with the discontinued legacy Single-Port Block Memory and Dual-Port Block Memory cores.

3.12.1 Features

- Optimized algorithms for minimum block RAM resource utilization or low power utilization
- Configurable memory initialization
- Optimized Verilog behavioral model for fast simulation times; structural simulation

models for precise simulation of memory behaviors.

- Selectable operating mode per port: WRITE_FIRST, READ_FIRST, or NO_CHANGE

3.12.2 Selectable Memory Algorithms

The core configures block RAM primitives and connects them together using one of the following algorithms: -

- Minimum Area Algorithm: The memory is generated using the minimum number of block RAM primitives. Both data and parity bits are utilized.

- Low Power Algorithm: The memory is generated such that the minimum number of block RAM primitives are enabled during a Read or Write operation.

- Fixed Primitive Algorithm: The memory is generated using only one type of block RAM primitive. For a complete list of primitives available for each device family, see the data sheet for that family.

3.13 Clocking Wizard

The Wizard guides us in setting the appropriate attributes for our clocking primitive, and allows us to override any wizard-calculated parameter. In addition to providing an HDL wrapper for implementing the desired clocking circuit, the locking Wizard also delivers a timing parameter summary for the circuit.

Features

- Frequency Synthesis: - allows output clocks to have different frequencies from the active input clock.

- Phase Alignment: - allows the output clock to be phase locked to a reference, such as the input clock pin for a device

- Minimize Power: - allows us to minimize the amount of power needed for the primitive. This is at the possible expense of frequency, phase offset, or duty cycle accuracy.

- Dynamic Phase Shift: - allows us to change the phase relationship on the output clocks.

- Minimize Output Jitter: - This feature minimizes the jitter on the output clocks.

- Safe Clock Start-up and Sequencing: - can be used to get a stable and valid clock at the output. It also enables clocks in a particular sequence order as specified in the configuration.

- Clock Monitor: - helps us to monitor the clock inputs to the Clocking Wizard. It can monitor up to four clocks.

3.14 Concat

The Concat IP core is used for concatenating bus signals of varying widths. The Concat IP core provides a mechanism to combine bus signals of varying width into a single bus. Using this block in the block design will not incur any resource usage of the FPGA. The input is an input bus with variable width. The number of input ports depends on the Number of Ports parameter. The “width” can be set to the desired number of bits. dout is the output port whose bit width equals the combined bit widths of all the input ports.

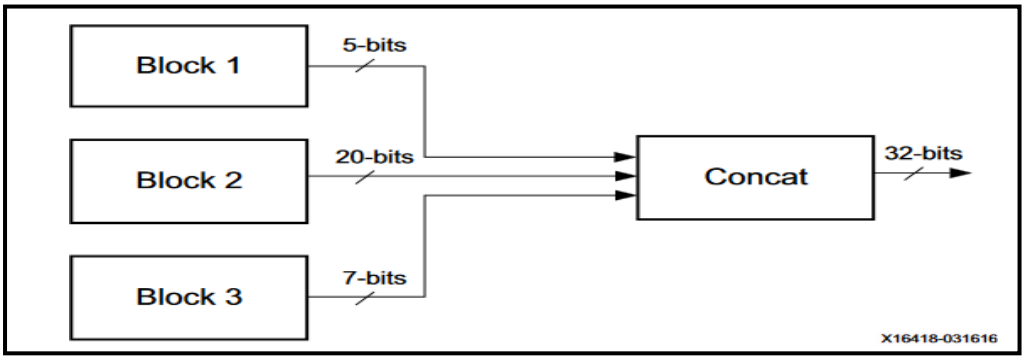

Figure 3.4: Concat IP in a System

Features

- Configurable number of ports

- Configurable bit width of each port

3.15 Conclusion

This chapter discusses the IP cores which were used in the circuitry. The chapter begins by defining the IP cores, then proceeds on to list out all the IP cores which have been utilized for the project. Further, detailed information regarding each IP cores along with their features and important properties/algorithms (if any) is also summarized.

Chapter 4

Software Description

4.1 Introduction

This chapter revolves around the two prominent software we used: -

- Xilinx Vivado ISE Design Suite
- WireShark

Xilinx Vivado ISE design suite is helpful in implementing the design and synthesis and also SDK launching and debugging.

Wireshark is a free and open-source packet analyzer. It is used for troubleshooting network analysis, software and communications protocol development. We are studying the packets received by the Ethernet via Wireshark.

4.2 Xilinx Vivado ISE Design Suite

4.2.1 Xilinx ISE

Xilinx ISE (Integrated Synthesis Environment) is a software tool produced by Xilinx for synthesis and analysis of HDL designs, enabling the developer to synthesize ("compile") their designs, perform timing analysis, examine RTL diagrams, simulate a design's reaction to different stimuli, and configure the target device with the programmer.

Xilinx ISE is a design environment for FPGA products from Xilinx, and is tightly-coupled to the architecture of such chips, and cannot be used with FPGA products from other vendors. The Xilinx ISE is primarily used for circuit synthesis and design, while ISIM or the ModelSim logic simulator is used for system-level testing. Other components shipped with the Xilinx ISE include the Embedded Development Kit (EDK), a Software Development Kit (SDK) and ChipScope Pro.

4.2.2 Xilinx Vivado

Vivado Design Suite is a software suite produced by Xilinx for synthesis and analysis of HDL designs, superseding Xilinx ISE with additional features for system on a chip development and high-level synthesis. Vivado represents a ground-up rewrite and re-thinking of the entire design flow (compared to ISE), and has been

described by reviewers as "well-conceived, tightly integrated, blazing fast, scalable, maintainable, and intuitive".

Like the later versions of ISE, Vivado includes the in-built logic simulator ISIM. Vivado also introduces high-level synthesis, with a tool chain that converts C code into programmable logic. Vivado has been described as a "state-of-the-art comprehensive EDA tool with all the latest bells and whistles in terms of data model, integration, algorithms, and performance".

4.2.3 User Interface

The primary user interface of the ISE is the Project Navigator, which includes the design hierarchy (Sources), a source code editor (Workplace), an output console (Transcript), and a processes tree (Processes).

The Design hierarchy consists of design files (modules), whose dependencies are interpreted by the ISE and displayed as a tree structure. For single-chip designs there may be one main module, with other modules included by the main module, similar to the main() subroutine in C++ programs. Design constraints are specified in modules, which include pin configuration and mapping.

The Processes hierarchy describes the operations that the ISE will perform on the currently active module. The hierarchy includes compilation functions, their dependency functions, and other utilities. The window also denotes issues or errors that arise with each function.

The Transcript window provides status of currently running operations, and informs engineers on design issues. Such issues may be filtered to show Warnings, Errors, or both.

4.2.4 Features and Components

Vivado was introduced in April 2012, and is an integrated design environment (IDE) with system-to-IC level tools built on a shared scalable data model and a common debug environment. Vivado includes electronic system level (ESL) design tools for synthesizing and verifying C-based algorithmic IP; standards-based packaging of both algorithmic and RTL IP for reuse; standards-based IP stitching and systems integration of all types of system building blocks; and the verification of

blocks and systems. A free version WebPACK Edition of Vivado provides designers with a limited version of the design environment.

- The **Vivado High-Level Synthesis** compiler enables C, C++ and SystemC programs to be directly targeted into Xilinx devices without the need to manually create RTL. Vivado HLS is widely reviewed to increase developer productivity, and is confirmed to support C++ classes, templates, functions and operator overloading. Vivado 2014.1 introduced support for automatically converting OpenCL kernels to IP for Xilinx devices. OpenCL kernels are programs that execute across various CPU, GPU and FPGA platforms.
- The **Vivado Simulator** is a component of the Vivado Design Suite. It is a compiled-language simulator that supports mixed-language, TCL scripts, encrypted IP and enhanced verification.

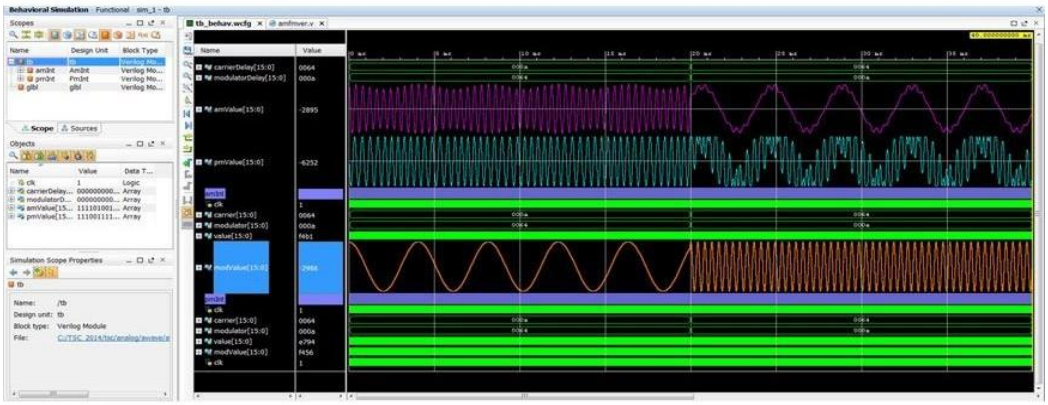

Figure 4.1: Vivado Simulator

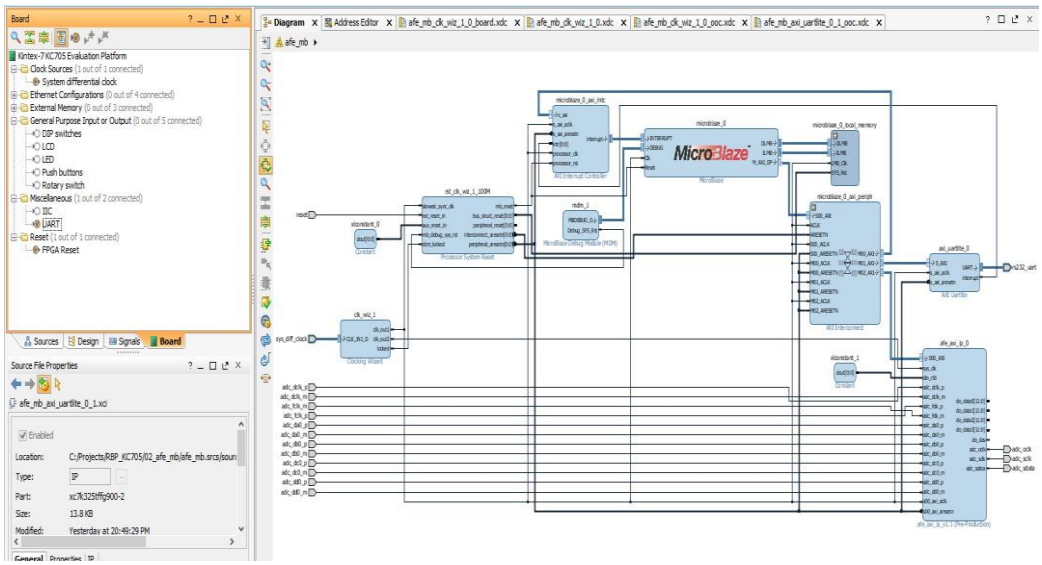

Figure 4.2: Vivado IP Integrator

- The **Vivado IP Integrator** allows engineers to quickly integrate and configure IP from the large Xilinx IP library. The Integrator is also tuned for MathWorks Simulink designs built with Xilinx's System Generator and Vivado High-Level Synthesis.
- The **Vivado TCL Store** is a scripting system for developing add-ons to Vivado, and can be used to add and modify Vivado's capabilities. TCL stands for Tool Command Language, and is the scripting language on which Vivado itself is based. All of Vivado's underlying functions can be invoked and controlled via TCL scripts.

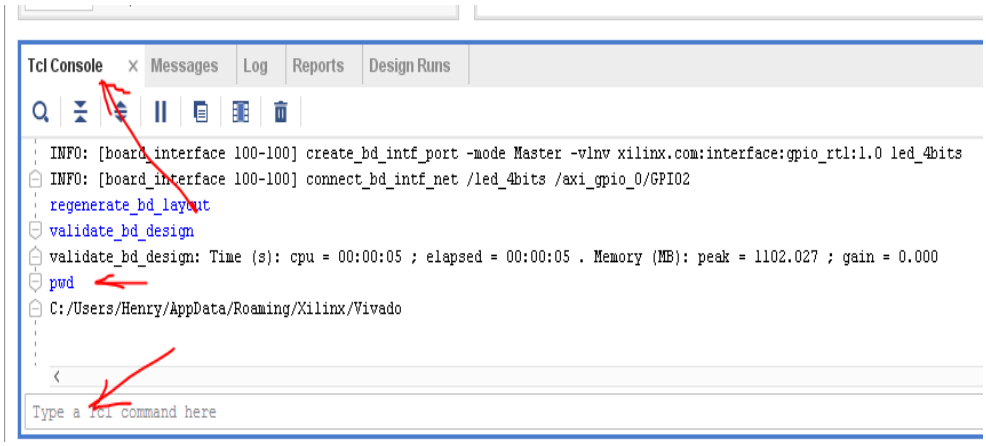

Figure 4.3: Vivado TCL Shell

4.2.5 Design Support

As of 2018, Xilinx recommends Vivado Design Suite for new designs with

- Ultrascale
- Ultrascale+
- Spartan-7
- Virtex-7
- Kintex-7
- Artix-7
- Zynq-7000.

Vivado supports newer high capacity devices, and speeds the design of programmable logic and I/O. Vivado provides faster integration and implementation

for programmable systems into devices with 3D stacked silicon interconnect technology, ARM processing systems, analog mixed signal (AMS), and many semiconductors intellectual property (IP) cores. Vivado is targeted at Xilinx's larger FPGAs, and is slowly replacing Xilinx ISE as Their mainline tool chain. As of 2014, Vivado covers Xilinx's mid-scale and large FPGAs, and ISE covered the mid-scale and smaller FPGAs and all CPLDs.

The design implementation in Vivado follows this following Procedure.

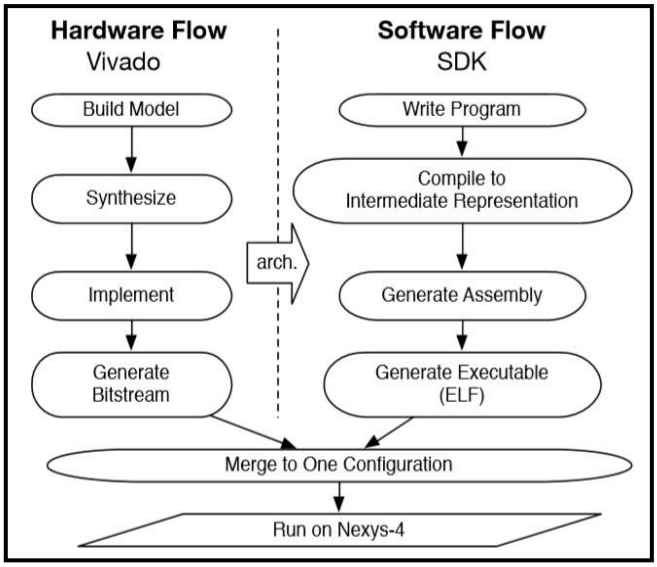

Figure 4.4: Overview of Design Flow

4.3 WireShark

Wireshark is cross-platform, using the Qt widget toolkit in current releases to implement its user interface, and using pcap to capture packets; it runs on Linux, macOS, BSD, Solaris, some other Unix-like operating systems, and Microsoft Windows. There is also a terminal-based (non-GUI) version called TShark. Wireshark, and the other programs distributed with it such as TShark, are free software, released under the terms of the GNU General Public License.

4.3.1 Functioning

Wireshark is very similar to tcpdump, but has a graphical front-end, plus some integrated sorting and filtering options. Wireshark lets the user put network interface controllers into promiscuous mode (if supported by the network interface controller), so they can see all the traffic visible on that interface including uni-cast traffic not sent to that network interface controller's MAC address.

However, when capturing with a packet analyzer in promiscuous mode on a port on a network switch, not all traffic through the switch is necessarily sent to the port where the capture is done, so capturing in promiscuous mode is not necessarily sufficient to see all network traffic.

Port mirroring or various network taps extend capture to any point on the network. Simple passive taps are extremely resistant to tampering. On GNU/Linux, BSD, and macOS, with libpcap1.0.0 or later, Wireshark 1.4 and later can also put wireless network interface controllers into monitor mode.

If a remote machine captures packets and sends the captured packets to a machine running Wireshark using the TZSP protocol or the protocol used by OmniPeek, Wireshark dissects those packets, so it can analyze packets captured on a remote machine at the time that they are captured.

It lets us see what's happening on our network at a microscopic level and is the de facto (and often de jure) standard across many commercial and non-profit enterprises, government agencies, and educational institutions. Wireshark development thrives thanks to the volunteer contributions of networking experts around the globe.

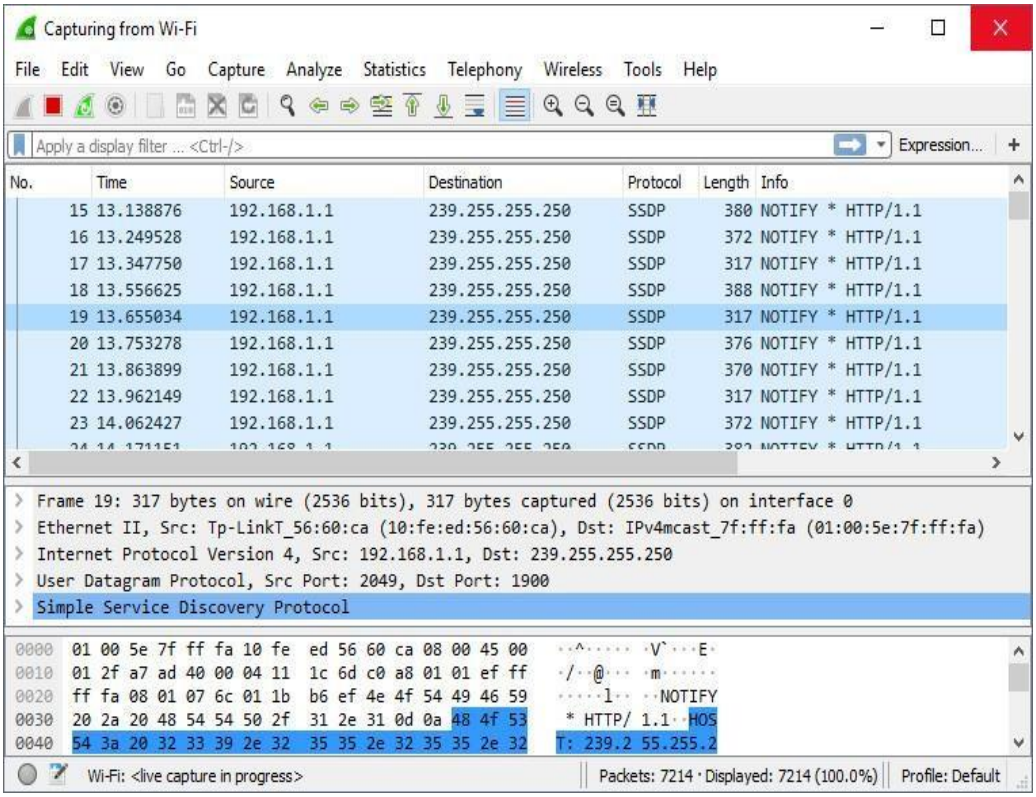

Figure 4.5: Packets received via Ethernet

4.3.2 Features

Wireshark is a data capturing program that "understands" the structure (encapsulation) of different networking protocols. It can parse and display the fields, along with their meanings as specified by different networking protocols. Wireshark uses pcap to capture packets, so it can only capture packets on the types of networks that pcap supports.

- Data can be captured "from the wire" from a live network connection or read from a file of already-captured packets.
- Live data can be read from different types of networks, including Ethernet, IEEE 802.11, PPP, and loop back.
- Captured network data can be browsed via a GUI, or via the terminal (command line) version of the utility, TShark.
- Captured files can be programmatically edited or converted via command-line switches to the "editcap" program.
- Data display can be refined using a display filter.
- Plug-ins can be created for dissecting new protocols.
- VoIP calls in the captured traffic can be detected. If encoded in a compatible encoding, the media flow can even be played.
- Raw USB traffic can be captured.
- Wireless connections can also be filtered as long as they traverse the monitored Ethernet.
- Various settings, timers, and filters can be set to provide the facility of filtering the output of the captured traffic.

Wireshark's native network trace file format is the libpcap format supported by libpcap and WinPcap, so it can exchange captured network traces with other applications that use the same format, including tcpdump and CA NetMaster. It can also read captures from other network analyzers, such as snoop, Network General's Sniffer, and Microsoft Network Monitor.

4.3.3 Security

Capturing raw network traffic from an interface requires elevated privileges on some platforms. For this reason, older versions of Ethereal/Wireshark and

tethereal/TShark often ran with super user privileges. Considering the huge number of protocol dissectors that are called when traffic is captured and recognizing the possibility of a bug in a dissector, a serious security risk can be posed.

Due to the rather large number of vulnerabilities in the past (of which many have allowed remote code execution) and developers' doubts for better future development, OpenBSD removed Ethereal from its ports tree prior to OpenBSD 3.6.

Elevated privileges are not needed for all operations. For example, an alternative is to run tcpdump or the dumpcap utility that comes with Wireshark with super user privileges to capture packets into a file, and later analyze the packets by running Wireshark with restricted privileges. To emulate near real time analysis, each captured file may be merged by merge cap into growing file processed by Wireshark. On wireless networks, it is possible to use the Aircrack wireless security tools to capture IEEE 802.11 frames and read the resulting dump files with Wireshark.

As of Wireshark 0.99.7, Wireshark and TShark run dumpcap to perform traffic capture. Platforms that require special privileges to capture traffic need only dumpcap run with those privileges. Neither Wireshark nor TShark need to or should be run with special privileges.

4.4 Using Vivado IDE andWireShark

4.4.1 Invoke the Vivado IDE and Create a Project

1. Open the Vivado IDE. From the Quick Start page, select Create New Project. The New Project wizard opens.

Figure 4.6: Vivado Quick Start Page

2.In the Project Name dialog box, type the project name and location.

3.In the Project Type dialog box, select RTL Project. Click Next.

4. In the Add Sources dialog box, ensure that the Target language is set to VHDL or Verilog. Leave the Simulator Language set to its default value of Mixed. Click Next.

5.In Add Existing IP dialog box, click Next.

6.In Add Constraints dialog box, click Next.

7.In the Default Part dialog box, select Boards and choose Kintex-7 Evaluation Board. Click End.

8.Review the project summary in the New Project Summary dialog box before clicking Finish to create the project.

4.4.2 Create an IP Integrator Design

1.From the Flow Navigator window (usually leftmost in Vivado), under IP Integrator item, select Create Block Design. The Create Block Design dialog box opens.

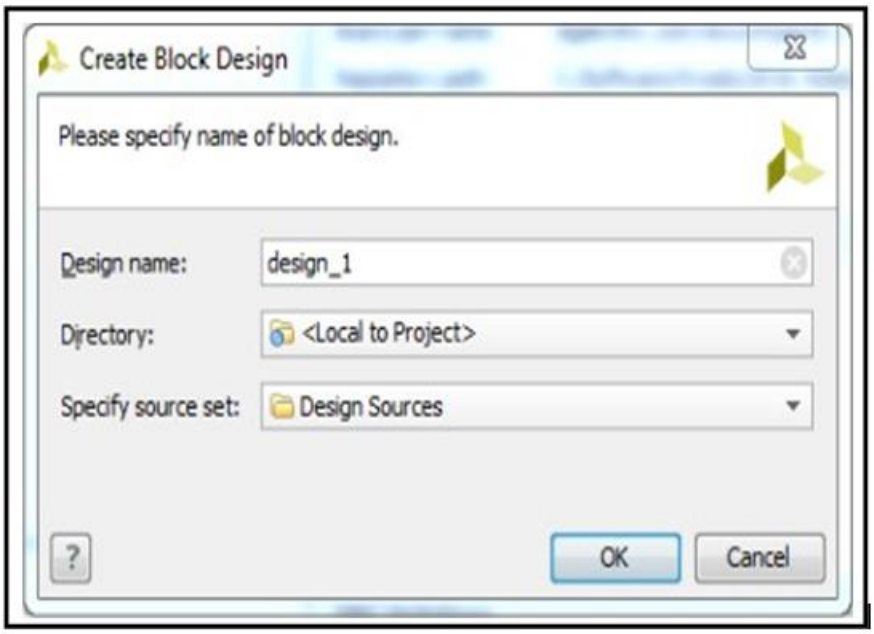

Figure 4.7: Name Block Design

2.Specify the IP subsystem design name. For this step, the tutorial will use the default value, but any name without spaces will do. Leave the Directory field set to its default value of <Local to Project>.

3.Leave Specify source set drop-down list set to its default value of Design Sources.

4.Click OK in the Create Block Design dialog box

5. We can either select Add IP here, or find the Add IP button on the left side of the Diagram tab (we will use this button when the design is not empty).
6. Type 'micro' in the Search field to find the IP, then select MicroBlaze and press the Enter key.

4.4.3 Use the Board Tab to Connect to Board Interfaces

1. Click the Board tab. We can see that there are several components listed in that tab. These components are present on the Kintex-7 board. They are listed under different categories in the Board window.
2. From the Board window, select System Clock from the Clock folder and drag and drop it into the block design canvas. This will allow us to use the 200MHz clock generated on board and add clocking logic to our design. (Since our designs are all synchronous, clocks will always be required!)
3. From the Board window, select Reset and drag and drop it into the block design canvas. This will allow us to use the Reset push button on the board to reset our system.
4. From the Board window, select UART Lite and drag and drop it into the block design canvas. This will allow us to communicate via the UART interface of the board with our design. This will be used to display simple text messages from our software running on our board, in a console connected with the board via serial.
5. From the board window, under Ethernet Configuration, select Onboard PHY. A Connect Board Component dialog box opens. Change the component mode to MII Mode. Select AXI Ethernet Lite.
6. In a similar fashion, include AXI Timer, AXI Interconnect, AXI Interrupt Controller, Processor System Reset and the Concat block for the interrupt controller.
7. Double click on the MicroBlaze block and set the Debug Module to Debug and UART. This will run the program step by step on the board.
8. Add Block Memory RAM (or BRAM) IP and a Block Memory Generator IP. Configure both to be dual ported.
9. Double click on the AXI Interconnect and configure it to have 4 Masters, for the Interrupt Controller, BRAM, Ethernet and UART respectively.

4.4.4 Connect the IP Cores

- Proceed by connecting output clock of the Clocking Wizard to the slaves and master clocks of the AXI Interconnect, MicroBlaze clock input, Ethernet clock input, UART clock input and to clock inputs for Processor System Reset and for the AXI Interrupt Controller.
- The external reset pin is connected to the ext_reset_in (i.e. the reset input) of the Processor System Reset. The output of this IP block has 5 resets which are:-

  bus_struct_reset: to be connected to the reset of the Local Memory Bus (LMB) of the MicroBlaze

  Interconnect_aresetn: to be connected to the ARESETN of the AXI Interconnect.

  peripheral_reset: reset pin to peripheral units.

  The peripheral_reset is active: High signal and the peripheral_aresetn is active-Low signals. This helps with signal loading and routing. In general, every peripheral can have its own peripheral_reset signal (peripheral_aresetn in case of peripherals connected to interconnect in a system). Since our circuit has an AXI Interconnect IP, therefore we need to leave peripheral_reset unconnected and connect the peripheral_aresetn to all the resets (both master and slave) of the AXI Interconnect.

- Connect the first slave (S00_AXI) to the M_AXI_DP (peripheral data interface) of the MicroBlaze. The second and the third slaves are of the M_AXI_DC and M_AXI_DI (i.e. the cache data interface and the cache instruction interface respectively) of the MicroBlaze respectively. The fourth slave slot is left unconnected in this design and can be considered an extra.
- There are 4 masters. The first, second, third and fourth masters are to be connected to the slaves of the Interrupt Controller, BRAM, Ethernet and UART respectively.
- Connect the corresponding resets of both masters and slaves present in the interconnect to their respective slave or master reset.
- Join the PORT_A and PORT_B pins of both BRAM and Block Memory blocks.

- Connect the interrupts of the UART and Ethernet to each of the inputs of the Concat IP. The output of this IP is given to the input of the Interrupt Controller.
- The Interrupt Controller interrupt and the MicroBlaze interrupt are connected.
- Join the Debug pin of the MicroBlaze input to the MDM output.

4.4.5 Validate Block Design

1. Click Validate Design button on the toolbar to make sure there are no critical warnings or errors in the design.

2. Save the design.

4.4.6 Synthesize the Design

1. In the Sources window, select the block design, right-click it and select Generate Output Products.

2. Under Sources, right-click the design and click Create HDL Wrapper

3. In the Flow Navigator, click on Run Synthesis.

4. After synthesis finishes, we can choose to have a look at the synthesised design, but finally we should Run Implementation in the Synthesis Completed dialog box. While the synthesis step refers to refining the design to a generic format (e.g. similar to software being compiled to an intermediate format), the implementation step generates a design for the specific FPGA on the board (e.g. generating the assembly code from the intermediate format). Click OK.

4.4.7 Take the Design through Implementation

The final result of the hardware design step is a bitstream - which is the low-level configuration for the target FPGA. In principle choosing Generate Bitstream in Flow Navigator will run through all the steps needed to generate this configuration. If we are running through the process as mentioned in the previous section, and we chose to run implementation directly, a new dialog box will open. Choose Generate Bitstream and click OK. To make an analogy with a software flow, if the implementation step generates the assembly code, the generate bitstream step creates the machine code. Note that this is still only the hardware to be emulated, but has no software to be run on it; this will be created later via SDK and incorporated in the bitstream as memory contents (if it’s located in the on-chip memory).

4.4.8 Exporting the Design to SDK

The software development kit needs to know about the underlying hardware architecture. If we just built our hardware, we can export it to SDK.

1. Connect a USB Type-A to Micro-B cable to the USB JTAG connector on the KC705 board. Power on the KC705 board.
2. Connect an Ethernet cable to the KC705.
3. Select File→Export→Export Hardware.
4. In Export Hardware dialog box, select Include bitstream check box, Make sure that the Export to field is set to <Local to Project>. Click OK.

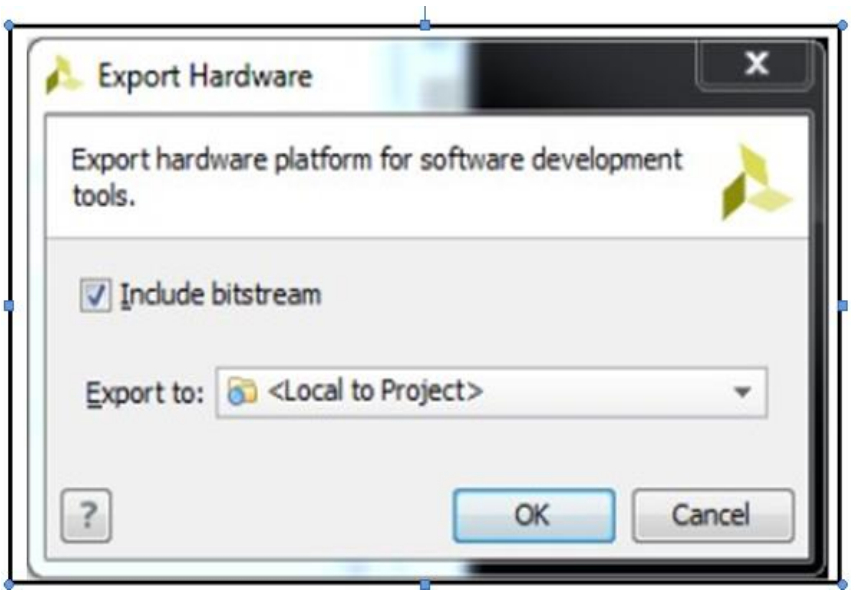

Figure 4.8: Export Hardware Dialog Box

5. Select File→Launch SDK. In the Launch SDK dialog box, make sure that both Exported location and Workspace drop-down lists are set to<Local to Project>.
6. In SDK, right-click mb_subsystem_wrapper_hw_platform_0 in the Project Explorer and select New→Project. In the New Project dialog box, select Xilinx Application Project. Click Next.
7. Type a name (such as per interest) for our project and choose standalone as OS platform. Click Next.
8. Select the Peripheral Tests application template, and click Finish.
9. Debug the code and run the code (if no errors). The UART output can be seen in the TeraTerm.

10. On the board, set SW11 to 1100 (1 = on, Position 1 → Position 4). This selects Gigabit Ethernet, with the packet generator off.
11. Open Wireshark. Set the filter address and select the COMM port associated with the devices.
12. The baud rate and frequency of the COMM port and that used in the design should match.
13. Right click on the Gigabit network connection and select Status. The status dialog will show the speed and number of packets. Wireshark should show no packets.

Figure 4.9: Status Dialog Box

14. Set SW11 to 1110 (1 = on, Position 1 → Position 4) for a moment to run the packet generator Set SW11 back to 1100. The status dialog shows a few packets received.
15. Wireshark captures and displays the actual packets. Use the arrow keys to move to the next packet.

16. Another method for the capturing of packets is to simply select the “Capture” button (green circle) in the WireShark to start the capture and select the “Stop” icon (red circle) instead of using the switches.

4.5 Conclusion

Using Xilinx Vivado ISE Design Suite, the implementation of the design UART to Ethernet conversion module is done. Xilinx have many distinct features which are very useful and user-friendly. Xilinx can reduce the effort of user in designing and implementing a design. Using WireShark, the converted data is received and checked.

Hardware architectures are created using Xilinx Vivado, a GUI that helps us to specify

- which processors, memory blocks and other soft IPs (peripherals) to use
- how the different IPs are interconnected
- the memory map, i.e. for addresses for memory mapped I/O peripherals
- how the different input/output signals map to actual pins on the FPGA and thus resources on the board

The output from Vivado is that part of the FPGA configuration that describes the hardware of our system. The FPGA and board resources require this configuration to emulate the hardware architecture that we described in Vivado.

Chapter 5

Implementation

5.1 Introduction

This chapter revolves around implementation of the project, i.e., creating a system (hardware and software). The simplest of programmable hardware architectures that can be built involves a single processor, a MicroBlaze in our case, and some minimal support for it (memory, interconnect). On top of this, the functionality for communicating over the UART is needed. Furthermore, without great effort, one can add functionality for sensing and controlling other physical components present on the board (such as push buttons, switches, temperature sensor, accelerometer).

Naturally, a programmable hardware architecture will require a program (software) to run on it. Therefore, the chapter will address the steps needed to build a simple software environment via Xilinx SDK. Finally, the last part of the chapter describes how to finally configure the FPGA with the hardware and software i.e. how to run our design and actually display the output on WireShark.

5.2 Block Diagram & Circuit Diagram

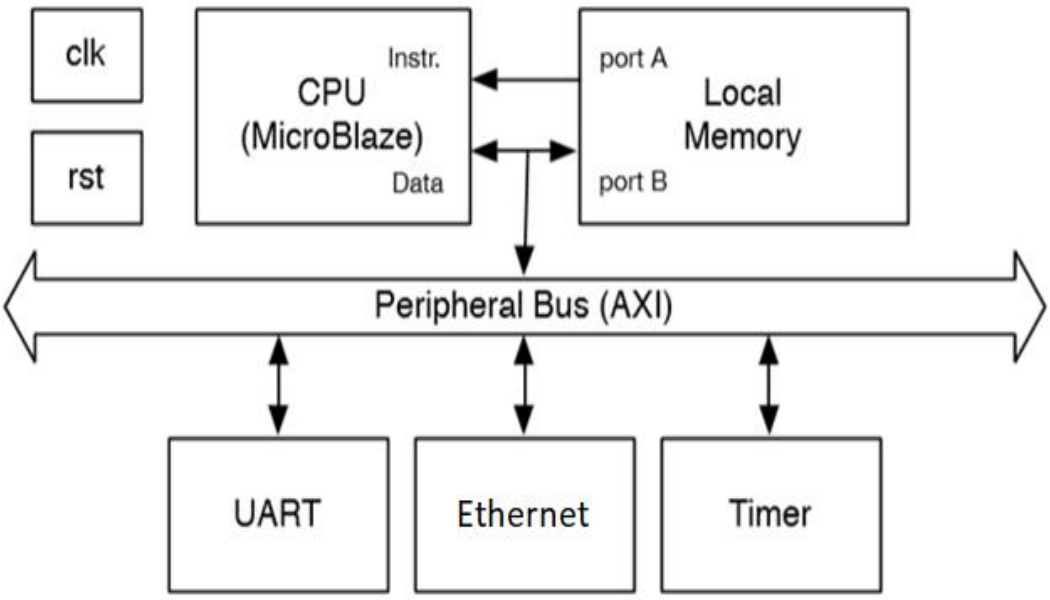

Figure 5.1: Block Diagram

Circuit Diagram

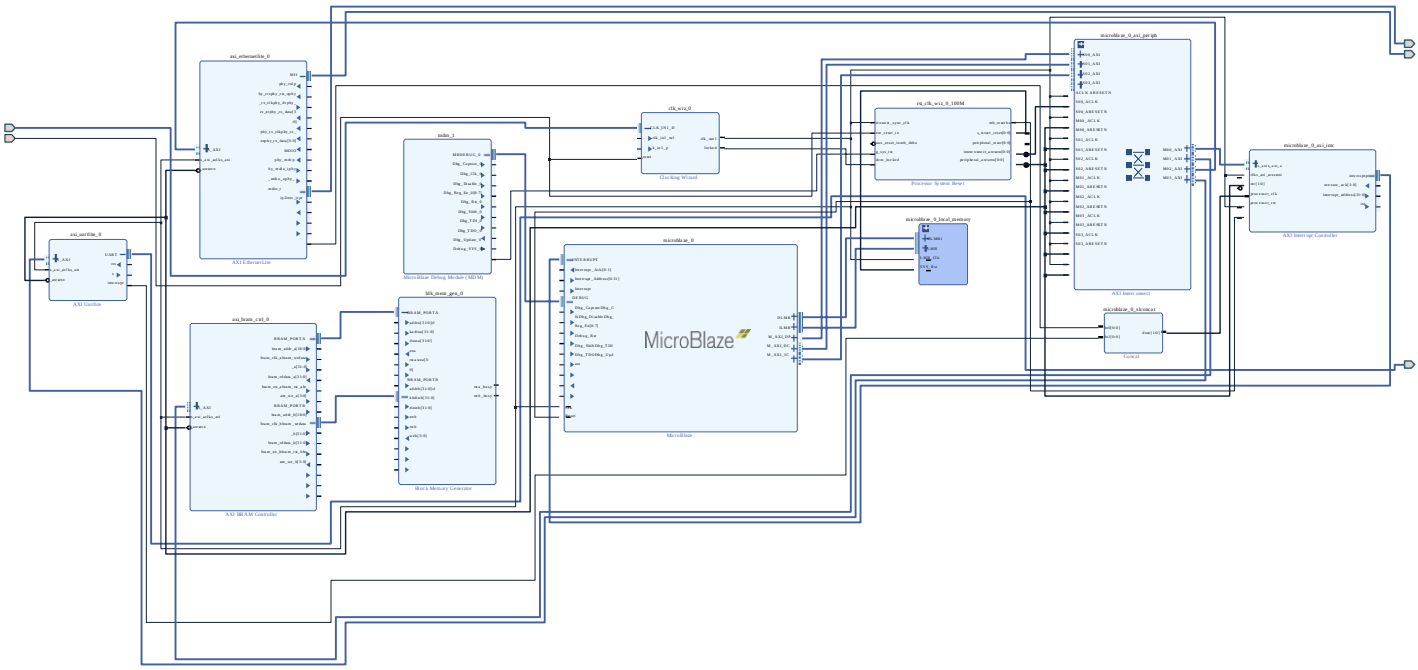

Figure 5.2: Circuit Diagram

5.3 Flow Chart and Algorithm

Flow Chart-1

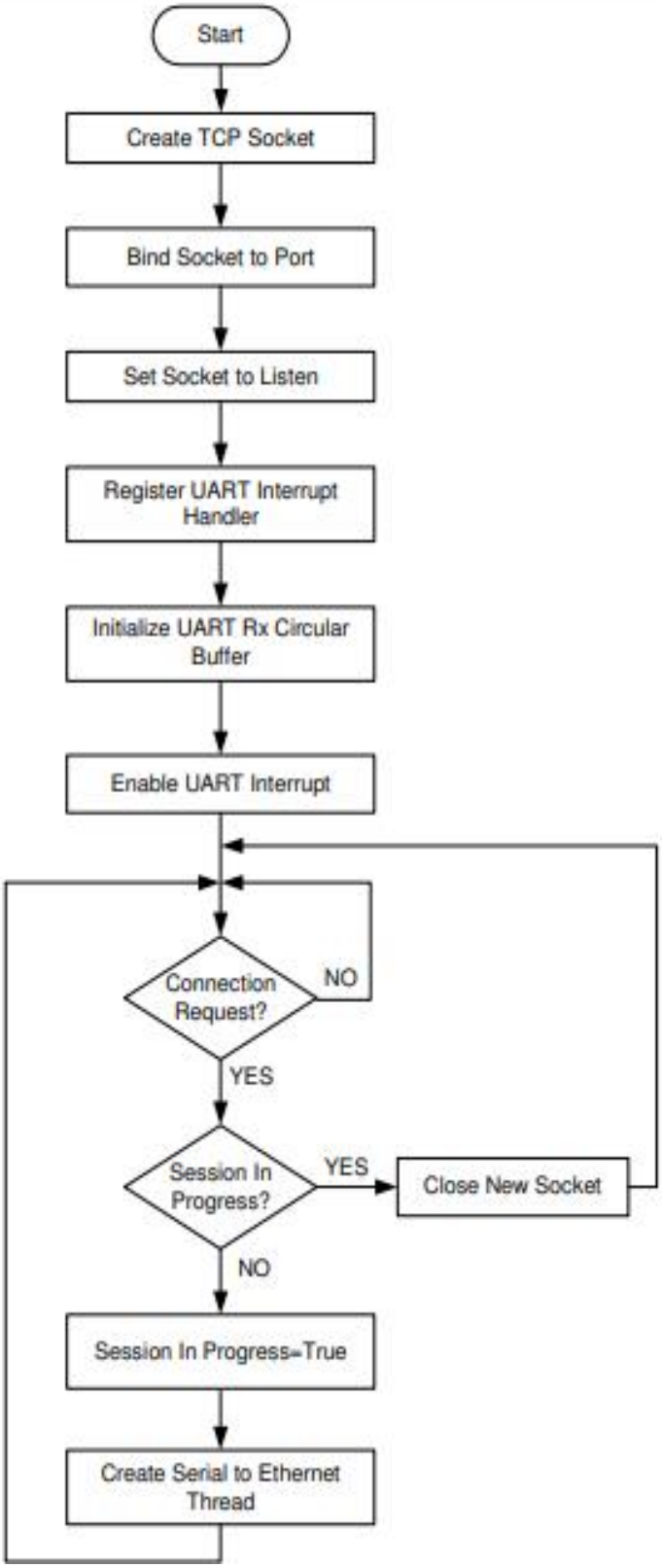

Figure 5.3: Serial to Ethernet Connection establishment Flow Chart

Algorithm-1

Step 1: First a TCP socket is created to provide an endpoint for communication.

Step 2: The TCP protocol is bound to a port number.

Step 3: The "listen" function call then turns the socket into a listening socket that can accept incoming connections.

Step 4: The UART interrupt handler is then registered with Xilkernel as it processes all the interrupts from the interrupt controller.

Step 5: Before UART interrupts are enabled, the UART Rx circular buffer structure is initialized with the correct length and location of the global buffer.

Step 6: Now the UART core interrupt and the UART interrupt in the interrupt controller are enabled so the UART interrupt handler may begin buffering data.

Step 7: A call is made to the BSD sockets "accept" function, which blocks until a connection request is made.

Step 8: Once a connection request has been made, the connection is accepted and a check to see if there is a Serial-to-Ethernet session in progress is made.

Step 9: If there is a Serial-to-Ethernet session in progress, the socket is closed as we can only have one Serial-to-Ethernet session currently.

Step 10: If there is no Serial-to-Ethernet session in progress, a new Serial-to-Ethernet thread is created to handle the request and the session in progress flag sets to true. The thread goes back to the connection requests.

A buffer contains data that is stored for a short amount of time, typically in the computer's memory (RAM). The purpose of a buffer is to hold data right before it isused. For example, when we download an audio or video file from the Internet, it may load the first 20% of it into a buffer and then begin to play. While the clip plays back, the computer continually downloads the rest of the clip and stores it in the buffer. Because the clip is being played from the buffer, not directly from the Internet, there is less of a chance that the audio or video will stall or skip when there is network congestion.

Flowchart-2

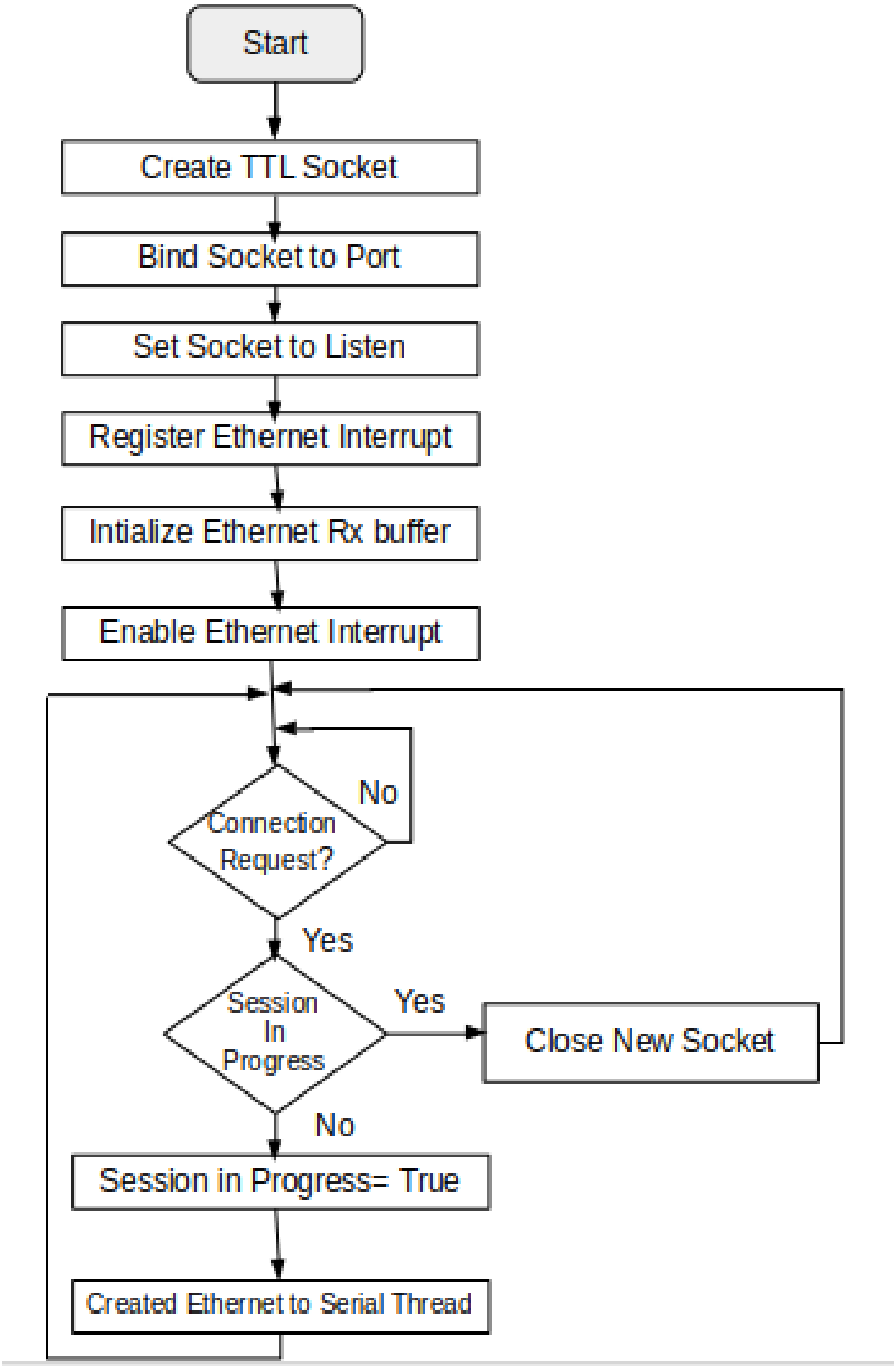

Figure 5.4: Ethernet to Serial Connection establishment Flow Chart

Algorithm-2

Step 1: First a TCP socket is created to provide an endpoint for communication.

Step 2: The TTL protocol is bound to a port number.

Step 3: The "listen" function call then turns the socket into a listening socket that can accept incoming connections.

Step 4: The Ethernet interrupt handler is then registered with Xilkernel as it processes all the interrupts from the interrupt controller.

Step 5: Before Ethernet interrupts are enabled, the Ethernet Rx circular buffer structure is initialized with the correct length and location of the global buffer.

Step 6: Now the Ethernet core interrupt and the Ethernet interrupt in the interrupt controller are enabled so the Ethernet interrupt handler may begin buffering data.

Step 7: A call is made to the BSD sockets "accept "function, which blocks until a connection request is made.

Step 8: Once a connection request has been made, the connection is accepted and a check to see if there is a Ethernet-to-Serial session in progress is made.

Step 9: If there is a Ethernet-to-Serial session in progress, the socket is closed as we can only have one Ethernet-to-Serial session currently.

Step 10: If there is no Ethernet-to-Serial session in progress, a new Serial-to-Ethernet thread is created to handle the request and the session in progress flag sets to true. The thread goes back to the connection requests.

5.4 Working Principle

5.4.1 UART to Ethernet Conversion Mechanism

When the clock input is given to the circuit the circuit starts to perform conversion. The UART sends request that it wants to write data into the BRAM. The request is forwarded to peripheral Interconnect which identifies the request from UART and forwards it to the interrupt controller. The interrupt controller forwards the interrupt to the MicroBlaze. MicroBlaze reads the instruction and allocate requested physical address via local memory, which maps this physical address into BRAM where the data write operation is done.

When Ethernet requests to read data from BRAM, the request is passed to interrupt controller, peripheral Interconnect then to MicroBlaze. The MicroBlaze reads the Instruction and allocates requested physical address via local memory which maps this physical address into BRAM where the data read operation is done. Hence, Serial data is being converted to Ethernet accessible data.

5.4.2 Ethernet to UART conversion Mechanism

When the clock input is given to the circuit the circuit starts to perform conversion. The Ethernet sends request that it wants to write data into the BRAM. The request is forwarded to peripheral Interconnect which identifies the request from Ethernet and forwards it to the interrupt controller. The interrupt controller forwards the interrupt to the MicroBlaze. MicroBlaze reads the instruction and allocate requested physical address via local memory, which maps this physical address into BRAM where the data write operation is done.

When UART requests to read data from BRAM, the request is passed to interrupt controller, peripheral Interconnect then to MicroBlaze. The MicroBlaze reads the Instruction and allocates requested physical address via local memory which maps this physical address into BRAM where the data read operation is done. Hence, Ethernet accessible data (packets) are converted to Serial data.

5.5 Advantages and Applications

5.5.1 Advantages

- Different work modes available: UDP Client, UDP Server, TCP Server, TCP Client.
- Possibility to adjust working model settings via a COM port or network.
- Supports 3.3V TTL level,RS232 (module products).
- Support of virtual COM port.
- Ensures reliable connection due to its exclusive heartbeat package mechanism, eliminating connect feign death.
- Packet broadcasting in UDP mode, has better anti-interference ability; through the gateway/switches/routers.
- Works in LAN/Internet (external network) and so on.

5.5.2 Applications

- Bi-direction transparent transmission between TTL and wired Ethernet.

Figure 5.5Application 1

- Extend the communication distance for serial devices.

Figure 5.6: Application 2

- Manage multi serial client devices remotely via one web server.

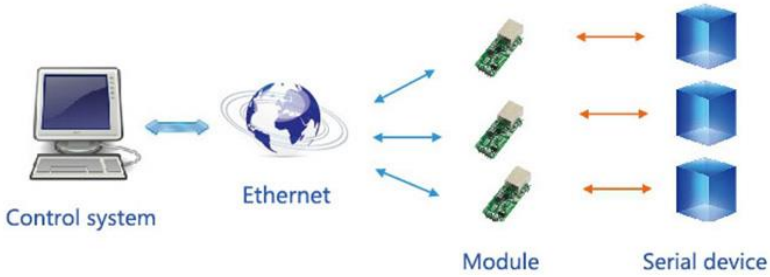

Figure 5.7: Application 3

- Manage serial server remotely via multi web clients

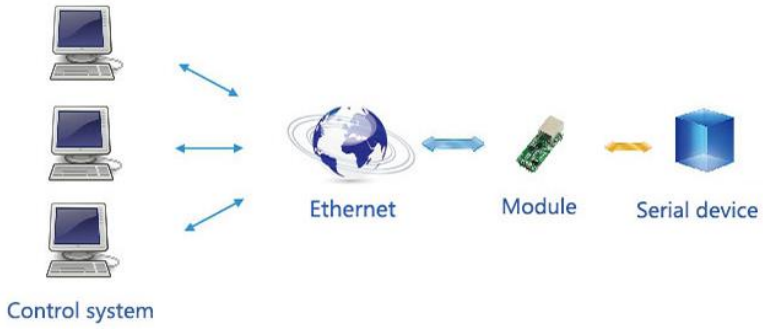

Figure 5.8: Application 4

5.6 Conclusion

The chapter lists out the implementation steps that were used to gather statistics about design timing, device utilization and power consumption. Also, this chapter discusses the steps to export the hardware architecture into SDK and create a simple application using this architecture in SDK. The last few steps make use of the WireShark software for the capturing of the packets.

Chapter 6

Results and Discussions

6.1 Introduction

This chapter consists outputs of Ethernet to serial conversion and Serial to Ethernet conversion. Each conversion has their respective application to view the result. These results are in application layer of the OSI model. The basic application to see output of Ethernet to Serial conversion is Tera Term and for Serial to Ethernet conversion is Wireshark. Tera Term is an open-source, free, software implemented, terminal emulator program.

6.2 Ethernet to Serial Conversion Output

Making a Connection

We should initially be presented with a "Tera Term: New connection" pop-up within the program. Here, we can select which serial port we'd like to open up. Select the "Serial" radio button. Then select port from the drop-down menu.

That'll open up the port. Tera Term defaults to setting the baud rate at 9600 bps (8-N-1). If we desire to adjust any serial settings, we can go to **Setup > Serial Port**.

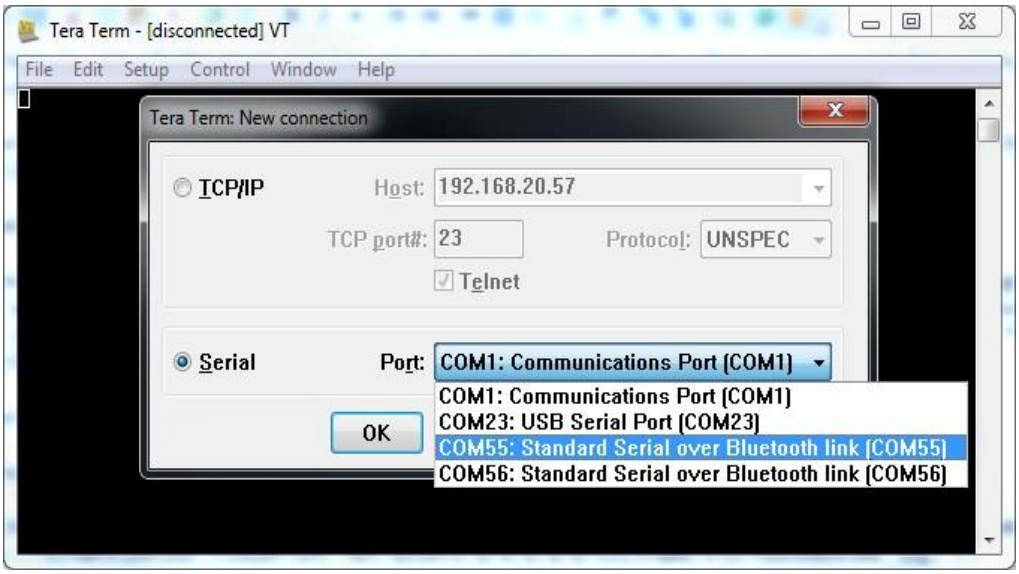

Figure 6.1: Creating a COM port

A window pops up similar to the one shown in above figure with a lot of serial port settings. Adjust what port we need and press "OK".

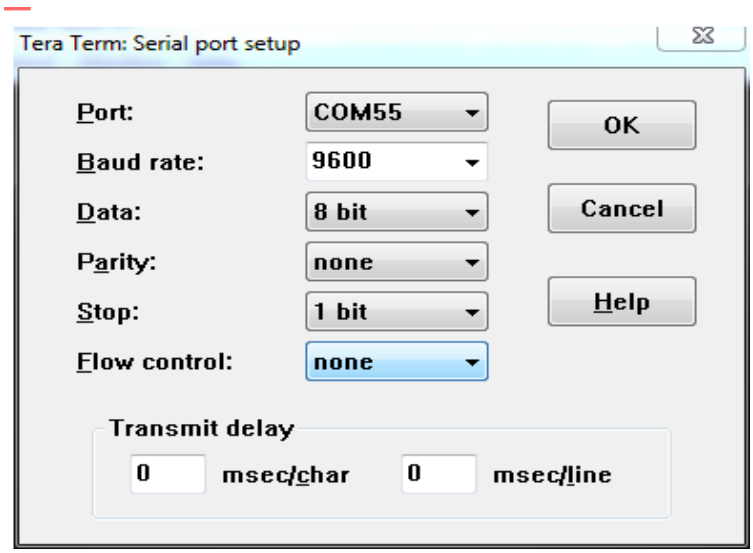

Figure 6.2: Tera Term Serial port setup window

The title of Tera Term window should change to something like "COM##:9600baud"

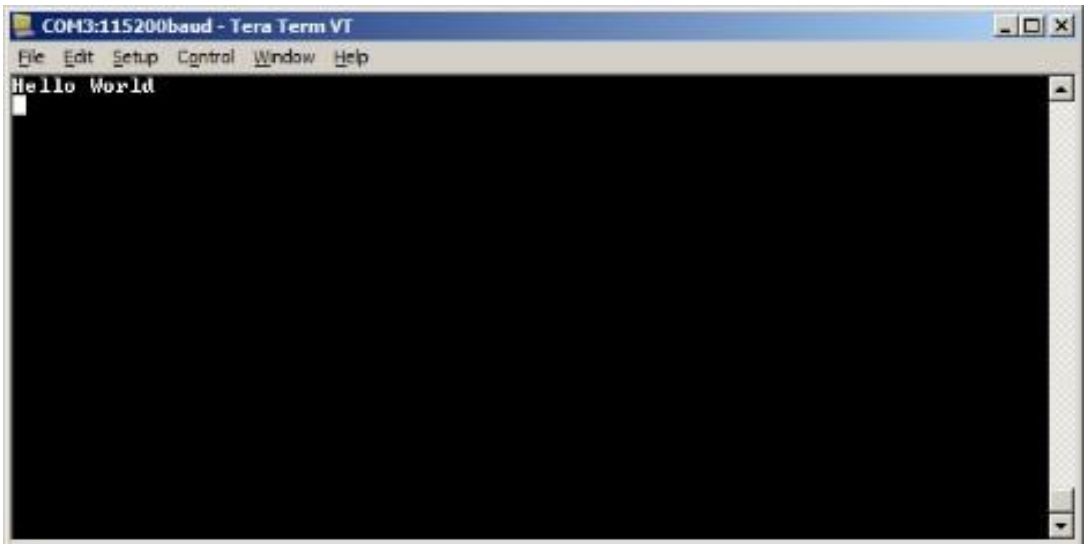

Figure 6.3: Message received via UART in Tera Term

The data sent from ethernet device is converted into serial port readable form. This information “Hello World” is displayed in the Tera term application as shown in the above figure.

6.3 Serial to Ethernet Conversion Output

Capturing Data Packets on Wireshark

When we open Wireshark, a screen that shows us a list of all of the network connections we can monitor will be seen. We also have a capture filter field, so we only capture the network traffic we want to see.

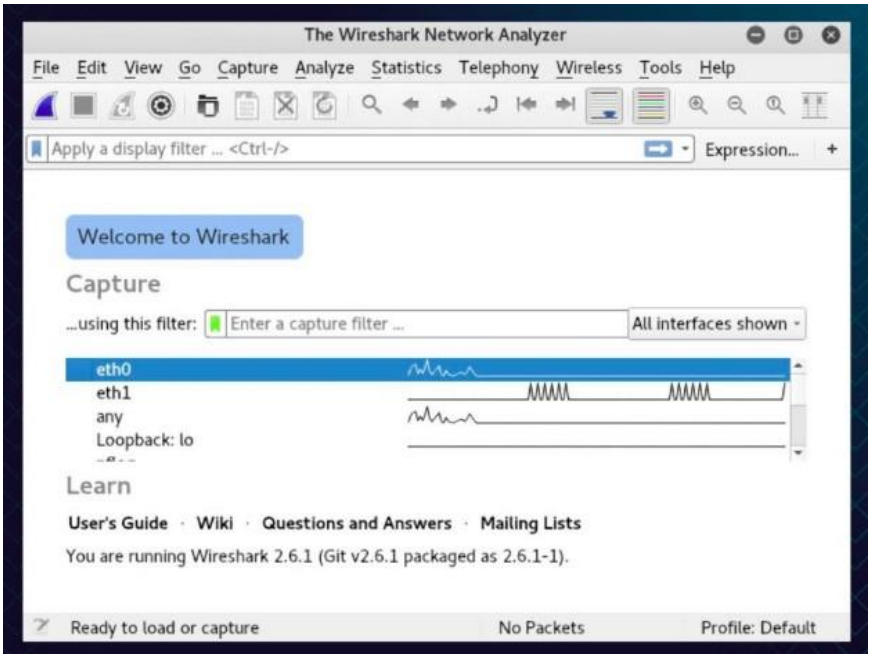

Figure 6.4: Wireshark Network Analyser

During the capture, Wireshark will show us the packets that it captures in real-time.

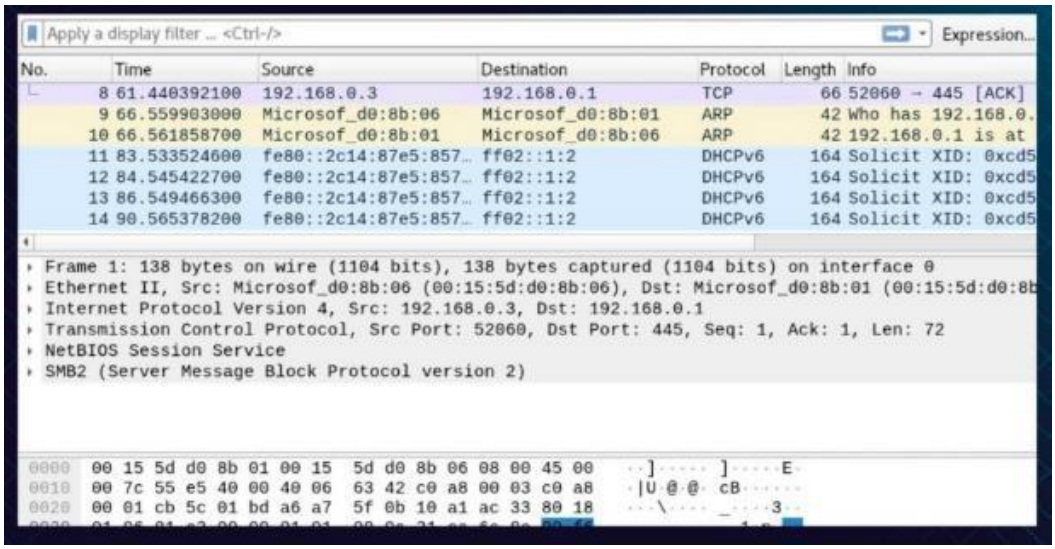

Figure 6.5: Packets captured via Ethernet

Wireshark shows three different panes for inspecting packet data. The Packet List, the top pane, is a list of all the packets in the capture. When we click on a packet, the other two panes change to show us the details about the selected packet. We can also tell if the packet is part of a conversation.

The top pane consists of the following columns explained below.

- **No.:** This is the number order of the packet that got captured. The bracket indicates that this packet is part of a conversation.
- **Time:** This column shows us how long after we started the capture that this packet got captured. We can change this value in the Settings menu if we need something different displayed.
- **Source:** This is the address of the system that sent the packet.
- **Destination:** This is the address of the destination of that packet.
- **Protocol:** This is the type of packet, for example, TCP, DNS, DHCPv6, or ARP.
- **Length:** This column shows us the length of the packet in bytes.
- **Info:** This column shows us more information about the packet contents, and will vary depending on what kind of packet it is.

The middle pane

The middle pane shows us as much readable information about the packet as possible, depending on what kind of packet it is. We can right-click and create filters based on the highlighted text in this field.

- **Frame 1:** Converts the frames to bytes.
- **Ethernet II:** Converts the source and destination addresses into hexadecimal format.(MAC addresses).
- **Internet Protocol Version 4:** Maps the MAC addresses of source and destination to IP addresses.
- **Transmission Control Protocol:** Receives packets from network layer and sends them to session service layer.
- **NetBIOS Service Layer:** Receives packets from TCP and sends it to presentation layer.
- **SMB2:** Service Message Block Protocol is used for file sharing on modern Windows hosts

The bottom pane

The bottom pane consists of Packet Bytes, displays the packet exactly as it got captured in hexadecimal.

When we are looking at a packet that is part of a conversation, we can right-click the packet and select follow to see only the packets that are part of that conversation.

6.4 Conclusion

Thus, the outputs of bi-directional conversion of Serial to Ethernet is observed. This type of conversion method is used for its reduced latency and Ensures reliable connection due to its exclusive heartbeat package mechanism, eliminating connect feign death.

Chapter 7

Conclusion and Future Scope

7.1 Conclusion

This project involves achieving UART over Ethernet access by way of the Serial to Ethernet Connector software application. This method of enabling access to the UART over LAN offers the benefit of not requiring any additional hardware. We can get UART to Ethernet connectivity without the need of any extra cabling, making it a more efficient solution.

A serial to Ethernet technology is an electronic appliance that is capable of converting data packets received over the TCP/IP network into serial data signals and back. The hardware redirects a serial device across the network, thus eliminating the need for its physical connection to a computer. Since there is no need to install additional serial communication line, with RS-232 to Ethernet device we can rely on the existing structure of our LAN.

The main purpose of this tool is to provide communication between serial devices and a computer via its COM ports that recognize the signals sent by a serial port over Ethernet. In practice, this means that using RS-232 to Ethernet hardware, we can access any serial device located within Ethernet (LAN) network from our machine, or manage a remote COM port peripheral over the Internet in case we use a Virtual Personal Network (VPN) or DNS.

Serial Port to Ethernet converters**,** gateways, and adapters are extremely useful when we are seeking to connect our computer's serial port to various console ports. This device allows communication to take place between a host PC and serial devices when located anywhere across an Ethernet network. The device can be applied in dispatching systems, commercial or technological automated metering systems, serial to IP communications and many other areas.

7.2 Future Scope

The integration of this project with more advanced technologies can uplift the services, applications and advantages of the system on whole.

- Using the feature of HDMI, i.e. High Definition Multimedia Interface. It is an all-digital audio-video interface which transmits signals in uncompressed format, e.g. uncompressed video data and compressed or uncompressed digital data from a HDMI-compliant source device to a computer monitor, video projector, digital television etc. Its significant benefit is to reduce the confusion of cables behind the device by a single cable only. As HDMI has combined both audio and video within one cable and give relief from the jungle of cables. Uncompressed digital audio-video gets transferred to the highest quality. For pictures and sound quality, there is no match of HDMI as it renders deep colors of 30-bit,36-bit and 48-bit. It supplies a superb sound quality without any interference of noise.
- The project can have an additional feature of PCIe. PCIe (peripheral component interconnect express) is an interface standard for connecting high-speed components. Every desktop PC motherboard has a number of PCIe slots we can use to add GPUs (aka video cards aka graphics cards), RAID cards, Wi-Fi cards or SSD (solid-state drive) add-on cards.

References

[1] J. Moon, and M. Yoon, "An implementation of a configurable serial-to-Ethernet converter using lwIP", Science International, vol. ED-11, pp. 34–39, Jan.2016.

[2] https://www.mootek.co.in/serial-to-ethernet-converters.html

[3] https://www.xilinx.com/products/design-tools/microblaze.html

[4] Flavius Gruian, "Xilinx Vivado/SDK Tutorial", Laboratory Session 1, EDAN15, March 21, 2017

[5] Jungho Moon, and Myunggon Yoon, "UART-to-Ethernet Converter", World Academy of Science, Engineering and Technology International Journal of Electronics and Communication Engineering Vol:11, No:12,2017

[6] Purushotham Reddy, and Dr S Nagakishore Bhavanam, "FPGA based for Low Power Multi Serial to Ethernet Gateway", International Conference on Innovation in Electronics and Communications Engineering (ICIECE-2017)